

Q.1 Answer the following Questions (1 Marks questions)

1.1 How many maximum memory locations and I/O devices can be addressed by an 8085 Microprocessor?

Memory Addressing:

- **Address Lines:** 16 (A0-A15).
- **Total Locations:** $2^{16} = 65,536$ locations.
- **Size:** 64 Kilobytes (KB).

I/O Addressing:

- **I/O Ports:** 256 distinct ports.
- **Address Range:** 00H to FFH (Hexadecimal).
- **Mechanism:** The 8085 uses a separate control signal (IO/M') to differentiate between memory and I/O accesses, allowing memory-mapped I/O (sharing the 64KB space) and I/O-mapped I/O (using the 256 I/O addresses).

1.2 Explain the use of HOLD and HLDA pins of 8085 microprocessor.

The HOLD and HLDA pins on the 8085 microprocessor manage Direct Memory Access (DMA) by allowing external devices to temporarily take control of the system bus, enabling faster data transfers without CPU intervention; **HOLD** (input) is the device's request for the bus, and **HLDA** (output) is the CPU's confirmation that it has released the buses for the device to use, facilitating efficient peripheral communication.

HOLD (Input Pin)

- **Function:** An external device (like a DMA controller) uses this input pin to signal the 8085 that it needs to use the system bus (Address & Data lines) for high-speed data transfer.
- **Action:** When an external device asserts a high signal on HOLD, it requests control of the bus.

HLDA (Hold Acknowledge) (Output Pin)

- **Function:** This is the 8085's response to a HOLD request, indicating it has relinquished control of the buses.
- **Action:** When the 8085 receives a HOLD request and is ready to give up the bus, it asserts a high signal on the HLDA pin, signaling the external device to begin its data transfer.

How They Work Together (DMA Cycle)

1. **Request:** An external device needing to transfer data (e.g., to/from memory) asserts a HIGH signal on the **HOLD** pin.
2. **Acknowledge:** The 8085 finishes its current operation, stops driving the address/data/control lines, and outputs a HIGH signal on the **HLDA** pin.
3. **Transfer:** The external device now has exclusive control of the system bus to perform its memory/IO data transfer.
4. **Release:** Once done, the external device deasserts the HOLD signal, and the 8085 resumes normal operation, taking back control of the buses.

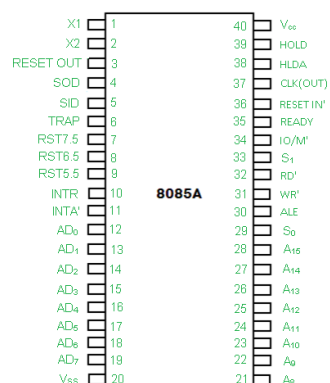
1.3 State the difference between op-code fetch (OF) and memory read (MR) cycles.

Key Differences Between Opcode Fetch (OF) and Memory Read (MR) Cycles:

Feature	Opcode Fetch (OF)	Memory Read (MR)
Cycle Type	First Machine Cycle ($M_1, M1$)	Subsequent Machine Cycle ($M_2, M_3 \dots M2, M3 \dots$)
Duration	4-6 T-states	3 T-states
Status (S_1, S_0)	$S_1 = 1, S_0 = 1, S1=1, S0=1$	$S_1 = 1, S_0 = 0, S1=1, S0=0$
Data Source	Program Counter (PC)	HL Pair / Immediate Address
Target Register	Instruction Register (IR)	Temporary/General Purpose Reg

1.4 State the importance of X1 and X2 pins of an 8085 microprocessor.

Pin diagram of 8085 microprocessor is shown below:



Key Pin Descriptions

1. Address Bus and Data Bus

Address Bus (A8 to A15): The address bus is unidirectional, i.e., bits flow in one direction from the microprocessor unit to the peripheral devices and uses the higher order address bus.

Address Data Bus (AD0 to AD7): These are bi-directional data pins used to transfer data between the microprocessor and memory or I/O devices. The microprocessor is an 8-bit processor, so it uses 8 data lines. These pins serve the dual purpose of transmitting lower order address and data byte. During 1st clock cycle, these pins act as lower half of address. In remaining clock cycles, these pins act as **data bus**.

2. Control and Status Signals

ALE - It is an Address Latch Enable signal. It goes high during first T state of a machine cycle and enables the lower 8-bits of the address, if its value is 1 otherwise data bus is activated.

IO/M' - It is a status signal which determines whether the address is for input-output or memory. When it is high(1) the address on the address bus is for input-output devices. When it is low(0) the address on the address bus is for the memory.

SO, S1 - These are status signals. They distinguish the various types of operations such as halt, reading, instruction fetching or writing.

IO/M'	S1	S0	Data Bus Status
0	1	1	Opcode fetch
0	1	0	Memory read
0	0	1	Memory write
1	1	0	I/O read
1	0	1	I/O write
1	1	1	Interrupt acknowledge
0	0	0	Halt

RD' - It is a signal to control READ operation. When it is low the selected memory or input-output device is read.

WR' - It is a signal to control WRITE operation. When it goes low the data on the data bus is written into the selected memory or I/O location.

READY - It senses whether a peripheral is ready to transfer data or not. If READY is high(1) the peripheral is ready. If it is low(0) the microprocessor waits till it goes high. It is useful for interfacing low speed devices.

3. Power Supply and Clock Frequency

Vcc - +5v power supply

Vss - Ground Reference

X1, X2 - A crystal is connected at these two pins. The frequency is internally divided by two, therefore, to operate a system at 3MHZ the crystal should have frequency of 6MHZ.

CLK (OUT) - This signal can be used as the system clock for other devices.

4. Interrupts and Peripheral Initiated Signals:

The 8085 has five interrupt signals that can be used to interrupt a program execution.

- (i) INTR
- (ii) RST 7.5
- (iii) RST 6.5
- (iv) RST 5.5
- (v) TRAP

The microprocessor acknowledges Interrupt Request by INTA' signal. In addition to Interrupts, there are three externally initiated signals namely RESET, HOLD and READY. To respond to HOLD request, it has one signal called HLDA.

1. **INTR (Interrupt Request)**: This pin is used to request an interrupt from an external device.
2. **RST7.5, RST6.5, RST5.5**: These are vectored interrupts with different priority levels.
3. **TRAP**: This is a non-maskable interrupt and has the highest priority.
4. **INTA (Interrupt Acknowledge)**: This pin is used to acknowledge interrupts from external devices.

5. Reset Signals

RESET IN - When the signal on this pin is low(0), the program-counter is set to zero, the microprocessor unit is reset.

RESET OUT - This signal indicates that the MPU is being reset. The signal can be used to reset other devices.

6. DMA Signals

HOLD - It indicates that another device is requesting the use of the address and data bus. Having received HOLD request the microprocessor relinquishes the use of the buses as soon as the current machine cycle is completed. Internal processing may continue. After the removal of the HOLD signal the processor regains the bus.

HLDA - It is a signal which indicates that the hold request has been received after the removal of a HOLD request, the HLDA goes low.

7. Serial I/O Ports

Serial transmission in 8085 is implemented by the two signals.

SID and SOD - SID is a data line for serial input whereas SOD is a data line for serial output.

Features of 8085 Microprocessor

1. **8-bit Processor:** Processes 8 bits of data at a time.
2. **16-bit Address Bus:** Can address up to 64 KB of memory.
3. **5 MHz Clock Speed:** Provides a decent processing speed for basic tasks.
4. **Low Power Consumption:** Ideal for embedded systems with minimal power needs.
5. **Interrupt System:** Features 5 interrupt pins for external signal response.

Limitations of 8085 Microprocessor

1. **Limited Memory Addressing:** Can address only up to 64KB of memory due to its 16-bit address bus.
2. **No Multiprocessing Support:** Cannot connect multiple processors for parallel processing.
3. **Limited Instruction Set:** Supports only 50 instructions, restricting its functionality for complex tasks.
4. **No Direct Memory Access:** Lacks DMA (Direct Memory Access) support, which slows data transfer between memory and peripherals.
5. **Slower Processing:** With a clock speed of up to 5 MHz, it is slower compared to modern processors.

1.5 What is an interrupt? Enlist the hardware interrupt sources (pins) available on the 8085 Microprocessor chip.

In the 8085 microprocessor, interrupt is a process in which control of the program transfers from the main program to the starting location defined by interrupt. It is a process by which some external device or peripheral informs microprocessor to become ready for data

communication by accepting the made request. Hence, it is a signal that temporarily suspends the normal execution of a program and redirects the control to a specific interrupt service routine (ISR). Interrupts allow the microprocessor to respond to external events, such as user input, system events, or hardware signals without the need for constant polling.

Types of Interrupt Signals in the 8085 Microprocessor

There are five interrupt signals in the 8085 microprocessor:

1. **TRAP:** The TRAP interrupt is a non-maskable interrupt that is generated by an external device, such as a power failure or a hardware malfunction. The TRAP interrupt has the highest priority and cannot be disabled.
2. **RST 7.5:** The RST 7.5 interrupt is a maskable interrupt that is generated by a software instruction. It has the second highest priority.
3. **RST 6.5:** The RST 6.5 interrupt is a maskable interrupt that is generated by a software instruction. It has the third highest priority.
4. **RST 5.5:** The RST 5.5 interrupt is a maskable interrupt that is generated by a software instruction. It has the fourth highest priority.
5. **INTR:** The INTR interrupt is a maskable interrupt that is generated by an external device, such as a keyboard or a mouse. It has the lowest priority and can be disabled.

When microprocessor receives any interrupt signal from peripheral(s) which are requesting its services, it stops its current execution and program control is transferred to a sub-routine by generating **CALL** signal and after executing sub-routine by generating **RET** signal again program control is transferred to main program from where it had stopped. When microprocessor receives interrupt signals, it sends an acknowledgement (INTA) to the peripheral which is requesting for its service.

Interrupts can be classified into various categories based on different parameters:

- **Hardware and Software Interrupts:** When microprocessors receive interrupt signals through pins (hardware) of microprocessor, they are known as *Hardware Interrupts*. There are 5 Hardware Interrupts in 8085 microprocessor. They are: *INTR, RST 7.5, RST 6.5, RST 5.5, TRAP* *Software Interrupts* are those which are inserted in between the program which means these are mnemonics of microprocessor. There are 8 software interrupts in 8085 microprocessor. They are: *RST 0, RST 1, RST 2, RST 3, RST 4, RST 5, RST 6, RST 7*.
- **Vectored and Non-Vectored Interrupts:** *Vectored Interrupts* are those which have fixed vector address (starting address of sub-routine) and after executing these, program control is transferred to that address. Vector Addresses are calculated by the formula $8 * \text{TYPE}$

INTERRUPT	VECTOR ADDRESS
TRAP (RST 4.5)	24 H
RST 5.5	2C H
RST 6.5	34 H
RST 7.5	3C H

For Software interrupts vector addresses are given by:

INTERRUPT	VECTOR ADDRESS
RST 0	00 H
RST 1	08 H
RST 2	10 H
RST 3	18 H
RST 4	20 H
RST 5	28 H
RST 6	30 H
RST 7	38 H

Non-Vectored Interrupts are those in which vector address is not predefined. The interrupting device gives the address of sub-routine for these interrupts. *INTR* is the only non-vectored interrupt in 8085 microprocessor.

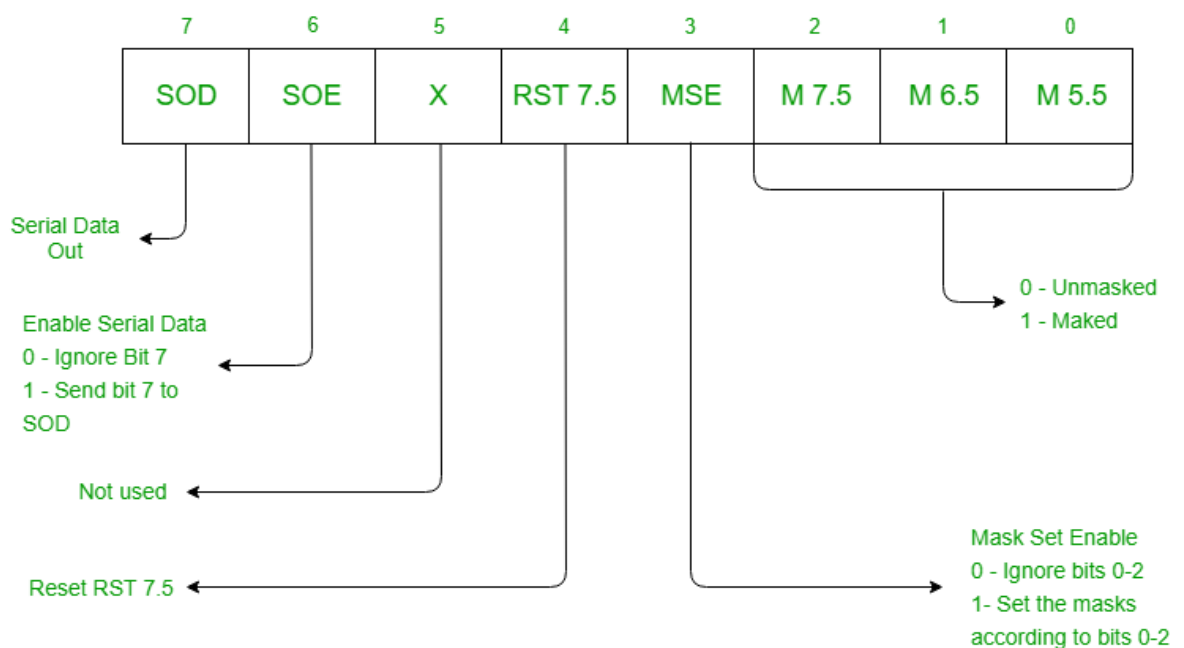
- Maskable and Non-Maskable Interrupts:** *Maskable Interrupts* are those which can be disabled or ignored by the microprocessor. These interrupts are either edge-triggered or level-triggered, so they can be disabled. *INTR*, *RST 7.5*, *RST 6.5*, *RST 5.5* are maskable interrupts in 8085 microprocessor. Non-Maskable Interrupts are those which cannot be disabled or ignored by microprocessor. *TRAP* is a non-maskable interrupt. It consists of both level as well as edge triggering and is used in critical power failure conditions.

Priority of Interrupts: When microprocessor receives multiple interrupt requests simultaneously, it will execute the interrupt service request (ISR) according to the priority of the interrupts.

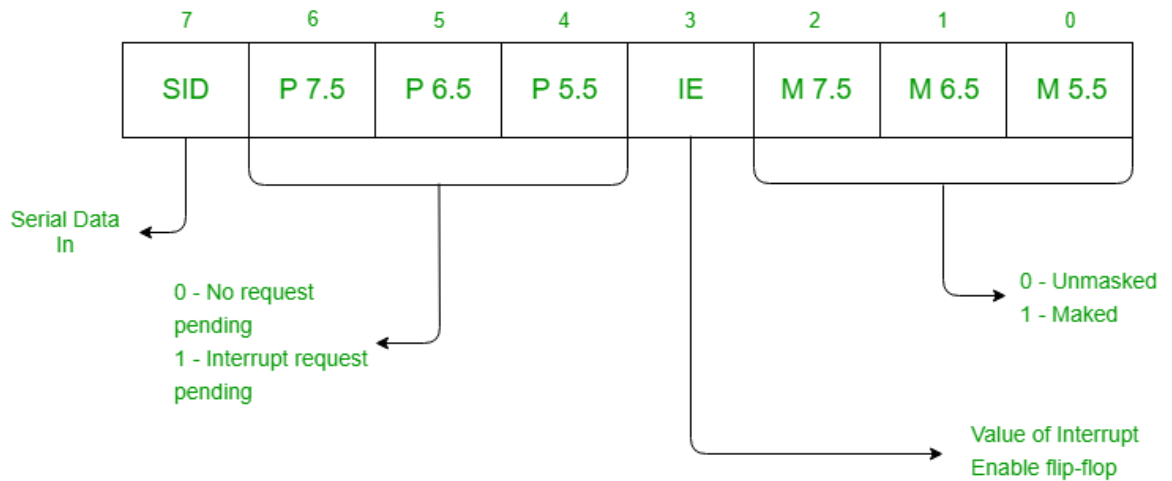


Instruction for Interrupts

- **Enable Interrupt (EI):** The interrupt enable flip-flop is set and all interrupts are enabled following the execution of next instruction followed by EI. No flags are affected. After a system reset, the interrupt enable flip-flop is reset, thus disabling the interrupts. This instruction is necessary to enable the interrupts again (except TRAP).
- **Disable Interrupt (DI):** This instruction is used to reset the value of enable flip-flop hence disabling all the interrupts. No flags are affected by this instruction.
- **Set Interrupt Mask (SIM):** It is used to implement the hardware interrupts (RST 7.5, RST 6.5, RST 5.5) by setting various bits to form masks or generate output data via the Serial Output Data (SOD) line. First the required value is loaded in accumulator then SIM will take the bit pattern from it.



- **Read Interrupt Mask (RIM):** This instruction is used to read the status of the hardware interrupts (RST 7.5, RST 6.5, RST 5.5) by loading into the A register a byte which defines the condition of the mask bits for the interrupts. It also reads the condition of SID (Serial Input Data) bit on the microprocessor.



Uses of Interrupts in 8085 microprocessor

- Interrupts in the 8085 microprocessor are used for various purposes, including:
- **Real-time processing:** Interrupts allow the microprocessor to respond quickly to external events in real-time, such as user input or hardware signals. This is particularly useful in applications such as control systems and data acquisition systems where time-critical operations are required.
- **Multi-tasking:** Interrupts enable the microprocessor to perform multiple tasks simultaneously by temporarily suspending the current task and executing the ISR for the interrupting event. This allows the microprocessor to switch between different tasks and maximize the utilization of system resources.
- **Input/output operations:** Interrupts can be used for handling input/output operations, such as data transfer between the microprocessor and external devices. This allows the microprocessor to perform other tasks while waiting for input/output operations to complete.
- **Error handling:** Interrupts can be used for error handling and exception handling, such as detecting and recovering from hardware or software errors.
- **Power management:** Interrupts can be used for power management, such as putting the microprocessor into a low-power mode when it is not needed and waking it up when an interrupt occurs.

1.6 Explain the use of READY pin of an 8085 microprocessor.

The READY pin in the 8085 microprocessor synchronizes the CPU with slower memory or I/O devices by pausing operations, ensuring data transfer happens at the peripheral's speed to prevent data loss. If the device is ready, it asserts READY high (1); if slow, it holds it low (0), forcing the CPU into a [wait state](#) (inserting clock cycles) until READY goes high, completing the read/write cycle.

How it works:

- **READY = High (1):**

The connected memory or I/O device is ready for data transfer; the CPU proceeds with the current machine cycle.

- **READY = Low (0):**

The device needs more time; the CPU halts (enters a wait state) and waits for the signal to go high before continuing the memory/IO read or write operation.

Key functions:

- **Synchronization:** Bridges the speed gap between the fast CPU and slower external devices.
- **Data Integrity:** Prevents data corruption or loss by ensuring data is exchanged only when both parties are ready.
- **Wait States:** Allows the CPU to insert extra clock cycles (wait states) to accommodate slower peripherals.

1.7 Explain RESET-IN AND RESET-OUT pin of an 8085 microprocessor.

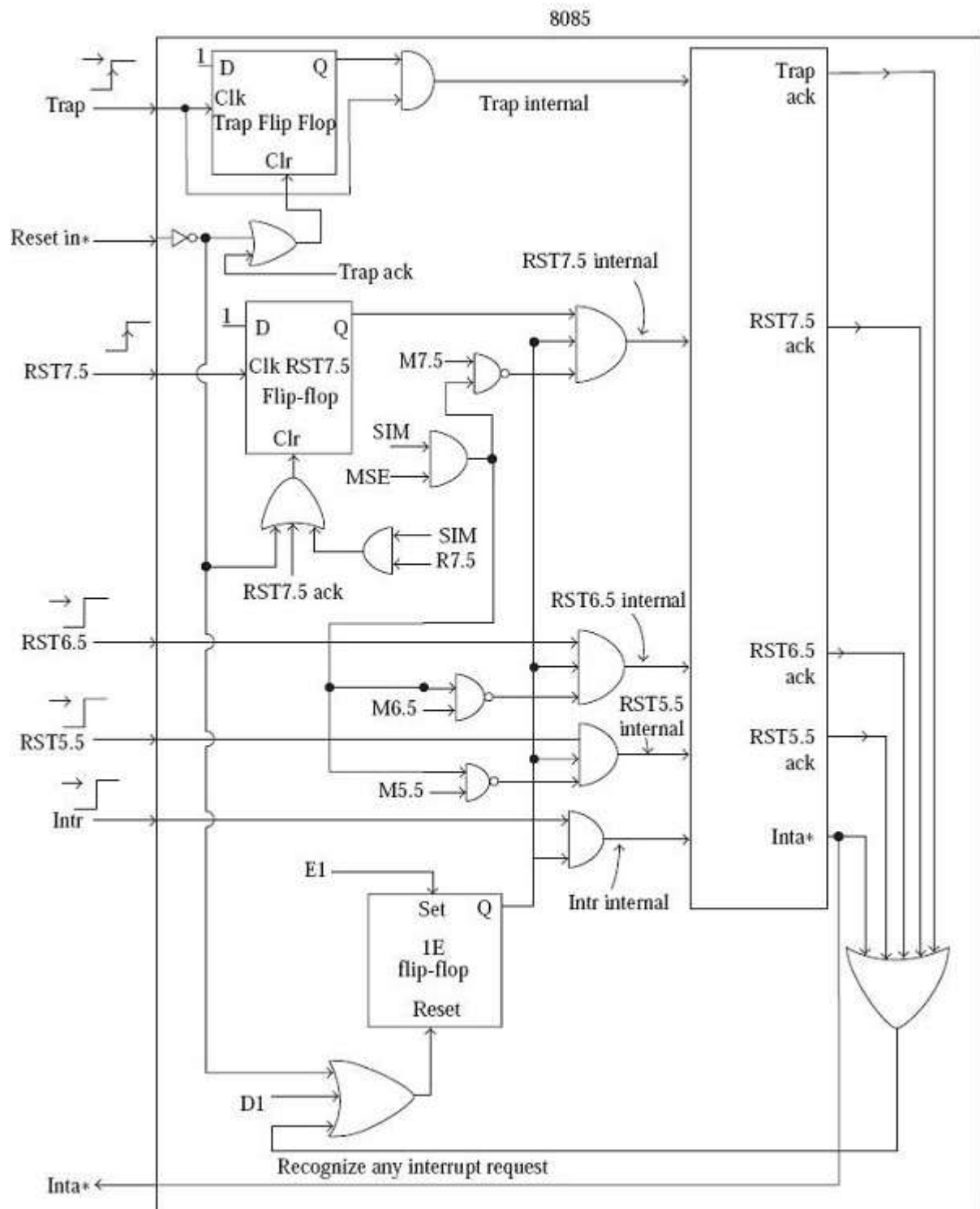
RESET IN - When the signal on this pin is low(0), the program-counter is set to zero, the microprocessor unit is reset.

RESET OUT - This signal indicates that the MPU is being reset. The signal can be used to reset other devices.

Intel 8085 consists of a RESET_IN* pin which is an active low input pin. We RESET 8085 by placing a logic 0 on this pin at least for 0.5μs, after that the power is supplied to Vcc pin of 8085. Moreover, in practice we place the RESET_IN* in logic 0 state for a few milliseconds. A typical reset circuit which we use in ALS 8085 kit, is shown in the following Fig.

The moment when the power supply is switched on, the Vcc pin gets +5V power here the RESET_IN* pin stays in logic 0 state for a time dependency on the RC time constant. In the circuit in following Fig., the time constant RC is about 50ms. Hence the RESET_IN* pin remains in logic 0 for a longer time than the minimum time which is 0.5μs.

When we already power and the system is in running condition sometimes we need to reset the 8085. We do not need to switch off and on for 8085.



1.8 What is Instruction Cycle?

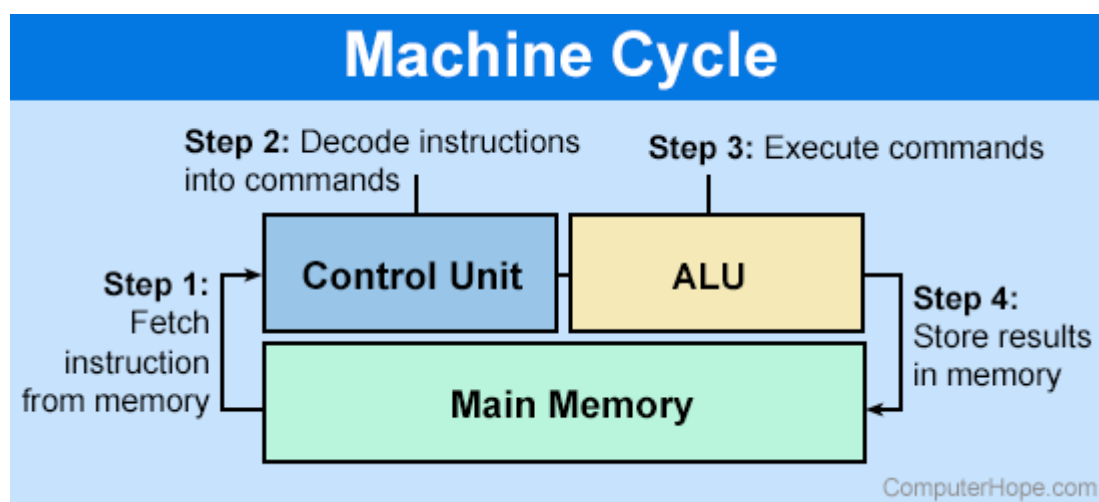
The [Instruction Cycle](#) (or [Fetch-Decode-Execute Cycle](#)) is the fundamental process a CPU follows to execute a program's instructions, consisting of fetching an instruction from memory, decoding it to understand the action, executing that action (like arithmetic or data transfer), and often storing the result before updating the [Program Counter](#) (PC) to fetch the next instruction. This continuous loop of fetching, decoding, and executing is how a computer performs its basic operations.

Here are the core steps in detail:

1. **Fetch:** The CPU retrieves the next instruction from main memory, using the address stored in the Program Counter (PC).
2. **Decode:** The instruction is sent to the Control Unit (CU), which interprets the binary code to figure out what operation to perform and what data is needed.
3. **Execute:** The CPU performs the required action, such as adding numbers, moving data, or accessing input/output devices.
4. **Store/Write Back (Optional/Implicit):** The result of the execution is written back to a register or memory location.
5. **Update PC:** The Program Counter is incremented to point to the next instruction in the sequence, and the cycle repeats.

1.9 What is Machine Cycle?

The steps performed by the computer [processor](#) for each machine language instruction received. The **machine cycle** is a four-process cycle that includes reading and interpreting the machine language, executing the code, and then storing that code.



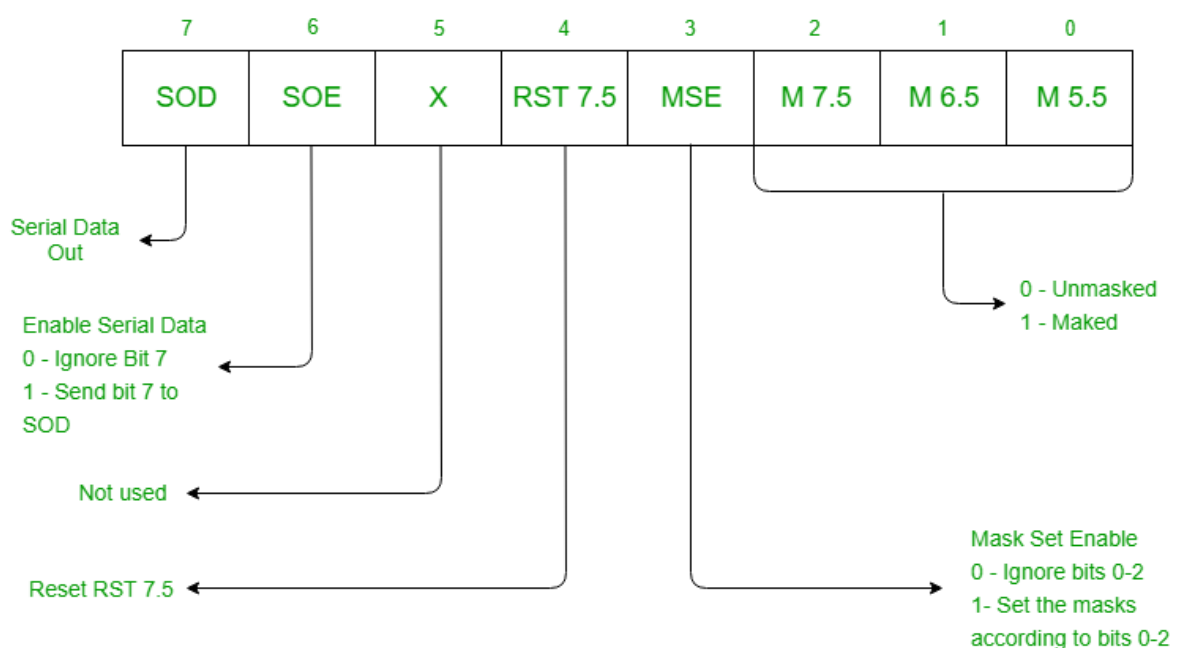
1.10 Which interrupt has the highest priority of 8085 Microprocessor?

When microprocessor receives multiple interrupt requests simultaneously, it will execute the interrupt service request (ISR) according to the priority of the interrupts.



1.11 What is SIM?

In 8085 Instruction set, SIM stands for “Set Interrupt Mask”. It is 1-Byte instruction and it is a multi-purpose instruction. The SIM instruction allows the microprocessor to selectively enable or disable interrupts by setting the appropriate bits in the Interrupt Mask Register (IMR). The IMR is a register that controls which interrupts are allowed to be processed by the [microprocessor](#). The SIM instruction takes one byte as an operand and sets the corresponding bits in the IMR based on the value of the operand.



Uses of SIM Instruction

- Masking/unmasking of RST7.5, RST6.5, and RST5.5
- Reset to 0 RST7.5 flip-flop.
- Perform serial output of data.

Advantages of SIM Instruction

- **Interrupt Control:** SIM is also used for better hand control of the hardware [interrupts](#) with this microprocessor comes with the ability to mask or unmask needed interrupts depending on the application.
- **Flexibility in Masking:** RST 7. 5, RST 6. 5, AND RST 5. 5 can be masked individually by SIM to meet the requirements of the interrupt system.
- **Serial Data Transmission:** SIM also has an application in serial output thus implements controlling the serial communication feature of the 8085 microprocessor.

Disadvantages of SIM Instruction

- **Complexity:** SIM instruction usage entails a correct application of interrupt of the 8085 and it may be slightly time-consuming for the beginner.
- **Limited Scope:** The SIM instruction only applies to hardware interrupts, for instances, the RST 7 interrupt. 5, RST 6. 5, and RST 5. 5 and also does not manage any other kind of interrupt.

1.12 Calculate address line for 4, 8, 16 byte memory.

- **4-byte memory**
 $\log_2(4) = 2$ address lines
- **8-byte memory**
 $\log_2(8) = 3$ address lines
- **16-byte memory**
 $\log_2(16) = 4$ address lines

1.13 What does microprocessor speed depends on?

- **Clock Speed/Rate:** This is the primary measure, representing the number of cycles per second the CPU can execute, usually measured in Gigahertz (GHz). A higher clock speed means the processor can perform more operations in a given time.
- **Architecture & Generation:** The design efficiency, including the instruction set (RISC vs. CISC) and pipeline design, dictates how many instructions can be executed per cycle (IPC). Newer generations generally offer better performance per clock cycle.

- **Number of Cores & Threads:** Modern processors use multiple cores to handle multiple tasks simultaneously (parallel processing), significantly improving speed for multi-threaded applications.
- **Cache Memory Size & Speed:** Larger L1, L2, and L3 caches reduce the time the CPU spends waiting for data from the slower main RAM, increasing the speed of data retrieval.
- **Bus Width:** The width of the data bus (e.g., 32-bit or 64-bit) determines how much data the processor can handle at once, directly affecting data transfer rates.
- **Thermal Management (Heat):** The ability to dissipate heat is a crucial limiting factor. Without adequate cooling, processors must throttle (slow down) to avoid damage.

Key Takeaway: While clock speed (GHz) is commonly cited, true performance is a combination of high clock speed, advanced architecture, large cache, multiple cores, and effective cooling.

Q.2 Describe the following questions. (2-3 Marks questions)

2.1 Explain Flag Register of 8085 Microprocessor.

In the 8085 microprocessor, the Flag Register is a kind of 8-bit register comprising five flags that indicate the results from arithmetic and logical operations. These flags include the Sign flag, indicating whether the result is negative; the Zero flag, set in case the result is zero; the Auxiliary Carry flag used in BCD operations; the Parity flag, which indicates whether the result has an even number of 1s; and the Carry flag, set in case a carry or borrow occurs. In essence, these flags assist the microprocessor in decision-making based on the result of operations.

The **Flag register** is a Special Purpose Register. Depending upon the value of the result after any arithmetic and logical operation, the flag bits become set (1) or reset (0). In 8085 microprocessor, the flag register consists of 8 bits and only 5 of them are useful. The 5 flags are:



1. **Sign Flag (S)** - After any operation if the MSB (D(7)) of the result is 1, it indicates the number is negative and the sign flag becomes set, i.e. 1. If the MSB is 0, it indicates the number is positive and the sign flag becomes reset i.e. 0. from 00H to 7F, sign flag is 0 from 80H to FF, sign flag is 1 1- MSB is 1 (negative) 0- MSB is 0 (positive)
Example: MVI A 30 (load 30H in register A) MVI B 40 (load 40H in register B) SUB B (A = A - B) These set of instructions will set the sign flag to 1 as 30 - 40 is a negative number. MVI A 40 (load 40H in register A) MVI B 30 (load 30H in register B) SUB B (A = A - B) These set of instructions will reset the sign flag to 0 as 40 - 30 is a positive number.
2. **Zero Flag (Z)** - After any arithmetical or logical operation if the result is 0 (00)H, the zero flag becomes set i.e. 1, otherwise it becomes reset i.e. 0. 00H zero flags is 1. from 01H to FFH zero flag is 0 1- zero-result 0- non-zero result **Example:** MVI A 10 (load 10H in register A) SUB A (A = A - A) These set of instructions will set the zero flag to 1 as 10H - 10H is 00H
3. **Auxiliary Carry Flag (AC)** - This flag is used in the BCD number system(0-9). If after any arithmetic or logical operation D(3) generates any carry and passes it on to D(4) this flag becomes set i.e. 1, otherwise, it becomes reset i.e. 0. This is the only flag register that is not accessible by the programmer 1-carry out from bit 3 on addition or borrows into bit 3 on subtraction 0-otherwise

Example: MVI A 2BH (load 2BH in register A) MVI 39H (load 39H in register B) ADD B ($A = A + B$) These set of instructions will set the auxiliary carry flag to 1, as on adding 2B and 39, the addition of lower-order nibbles B and 9 will generate a carry.

4. **Parity Flag (P)** - If after any arithmetic or logical operation the result has even parity, an even number of 1 bit, the parity register becomes set i.e. 1, otherwise it becomes reset i.e. 0. 1-accumulator has an even number of 1 bits 0-accumulator has odd parity

Example: MVI A 05 (load 05H in register A) This instruction will set the parity flag to 1 as the BCD code of 05H is 00000101, which contains an even number of ones i.e. 2.

5. **Carry Flag (CY)** - Carry is generated when performing n bit operations and the result is more than n bits, then this flag becomes set i.e. 1, otherwise, it becomes reset i.e. 0. During subtraction ($A-B$), if $A > B$ it becomes reset, and if $(A < B)$ it becomes set. Carry flag is also called the borrow flag. 1-carry out from MSB bit on addition or borrow into MSB bit on subtraction 0-no carry out or borrow into MSB bit.

Example: MVI A 30 (load 30H in register A) MVI B 40 (load 40H in register B) SUB B ($A = A - B$) These set of instructions will set the carry flag to 1 as $30 - 40$ generates a carry/borrow. MVI A 40 (load 40H in register A) MVI B 30 (load 30H in register B) SUB B ($A = A - B$) These set of instructions will reset the carry flag to 0 as $40 - 30$ does not generate any carry/borrow.

The Flag register is a 8-bit register in the 8085 microprocessor that contains information about the status of the arithmetic and logic operations performed by the processor. The bits in the Flag register are used to indicate whether the result of an operation is zero, positive, negative, or if there was a carry or borrow during the operation.

The Flag register is used by the processor to make decisions based on the results of arithmetic and logic operations, such as branching to a different instruction based on the value of a flag.

Uses of Flag Register

- **Conditional branching:** The flags in the Flag register are used to make decisions about whether to branch to a different instruction or not. For example, the Jump on Zero (JZ) instruction checks the Zero Flag to see if the result of the previous operation was zero. If it was, the program counter is changed to jump to a different address.
- **Arithmetic operations:** The Flag register is used extensively during arithmetic operations such as addition, subtraction, multiplication, and division. The Carry Flag is used to detect if there was a carry during addition or a borrow during subtraction. The Auxiliary Carry Flag is used to detect if there was a carry from the 3rd bit to the 4th bit during addition or a borrow from the 4th bit to the 3rd bit during subtraction.

- **Logical operations:** The Flag register is also used during logical operations such as AND, OR, and XOR. The Zero Flag is used to detect if the result of the logical operation was zero. The Sign Flag is used to detect if the result was negative.
- **System calls:** The Flag register is used by the operating system to determine the status of the CPU before and after a system call. For example, the operating system might check the Carry Flag to see if there was a carry during an arithmetic operation, and then take appropriate action based on the result.

Advantages

- **Efficient error detection:** The flag register can be used to efficiently detect errors and other conditions during program execution. For example, the carry flag can be used to detect overflow or underflow during arithmetic operations.
- **Simplifies branching:** The flags can be used to make decisions about branching and other control flow operations. For example, the zero flag can be used to determine if a value is zero, and the sign flag can be used to determine if a value is negative.
- **Saves memory:** The flag register can help reduce the amount of memory required for storing status information.
- **Faster execution:** By using flags instead of performing additional operations to check conditions, the 8085 microprocessor can execute programs more quickly.

Disadvantages

- **Limited information:** The flag register provides a limited set of status information that may not be sufficient for all applications.
- **Limited control:** The flags in the flag register are set or cleared automatically by the processor and cannot be directly manipulated by the programmer.
- **Complexity:** The use of flags can add complexity to programming and debugging, particularly when multiple flags are used in conjunction with one another.
- **Overwriting:** Since the flag register is also used for other purposes, such as storing the stack pointer, the status flags can be accidentally overwritten during program execution.

Applications of 8085 Microprocessor

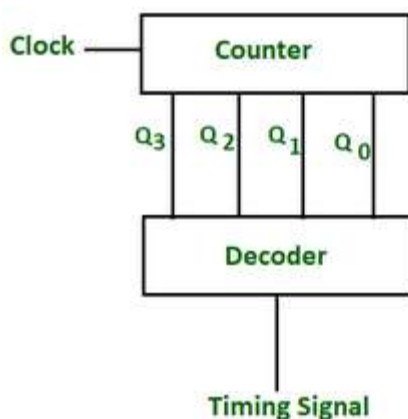
A few of the applications of the microprocessor 8085 involve the following:

- **Embedded Systems:** Many embedded systems used 8085 for applications such as control system, instrumentation, and communications.
- **Education:** The 8085 has widely found applications as teaching in courses of microprocessors due to its simple architecture.

- **Traffic Light Control Systems:** It finds application in simple automation tasks, for example, controlling the lights of traffic or another sort of sequencing applications.
- **Home Appliances:** From washer and dryer to microwave oven, this small appliance of microprocessor found its home. Industrial Automation: It played a major role in controlling machinery and automation systems of industries during its time.

2.2 Explain Controlling signal generating logic circuit.

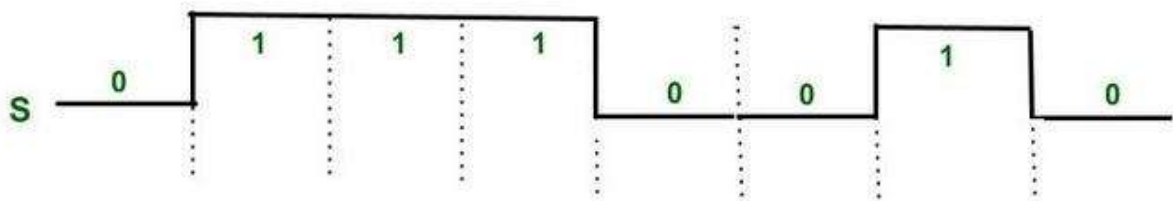
- In various digital applications(For example : hardwired control unit) control signals are needed to start, execute and step various operations in a particular time sequence.
- For this control signals are required and for the generation of control signals, a counter circuit is designed whose outputs are connected to a decoder. The decoder provides the required control signal.
- The counter can be synchronous or asynchronous.
- The procedure for designing the counter is the same refer ([this](#)).



Block diagram for control signal generation

The design of control signal can be understood by considering the example.

Example - Generate a control signal which can deliver the following pulse train. The pulse train will repeat after 7 pulses.



Control Signal

Here we need to generate a control signal (say S) which can generate periodic pulse train of **0111001** and then repeats. The pulse train repeats after 7 seven pulses. Therefore, **mod - 7 counter is needed**. The outputs of Mod-7 counter will be connected to a decoder circuit. For this a mod –7 counter is to be designed. Three T flips are required(because we need to count up to 000 to 110, therefore 3 bits are needed).

For the design for Mod — N counter please refer [this article](#).

Design for mod-7 counter -

Here Q is previous state and Q* is next state.

Q_2	Q_1	Q_0	Q_2^*	Q_1^*	Q_0^*	T_2	T_1	T_0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	0	0	0	1	1	0

Circuit excitation table for mod- 7 Counter

The expressions for inputs of T flip-flops obtained from the K – maps shown below.

$Q_2 \backslash Q_1 Q_0$		00	01	11	10
		0	0	1	0
	1	0	0	X	1

$$T_2 = Q_1 Q_0 + Q_2 Q_1$$

$Q_2 \backslash Q_1 Q_0$		$Q_1 Q_0$			
		00	01	11	10
0	0	0	1	1	0
1	1	0	1	X	1

$$T_1 = Q_0 + Q_2 Q_1$$

$Q_2 \backslash Q_1 Q_0$		$Q_1 Q_0$			
		00	01	11	10
0	1	1	1	1	1
1	1	1	X	0	

$$T_0 = Q_2' + Q_1'$$

The truth table for the **decoder is obtained by observing of the given timing sequence(0111001)**. The simplified expression for the output S of decoder is obtained using the K-map. The not used counts are considered as don't care. Each counting sequence is mapped to one control signal bit. The combinational logic of decoder can be found by solving K map.

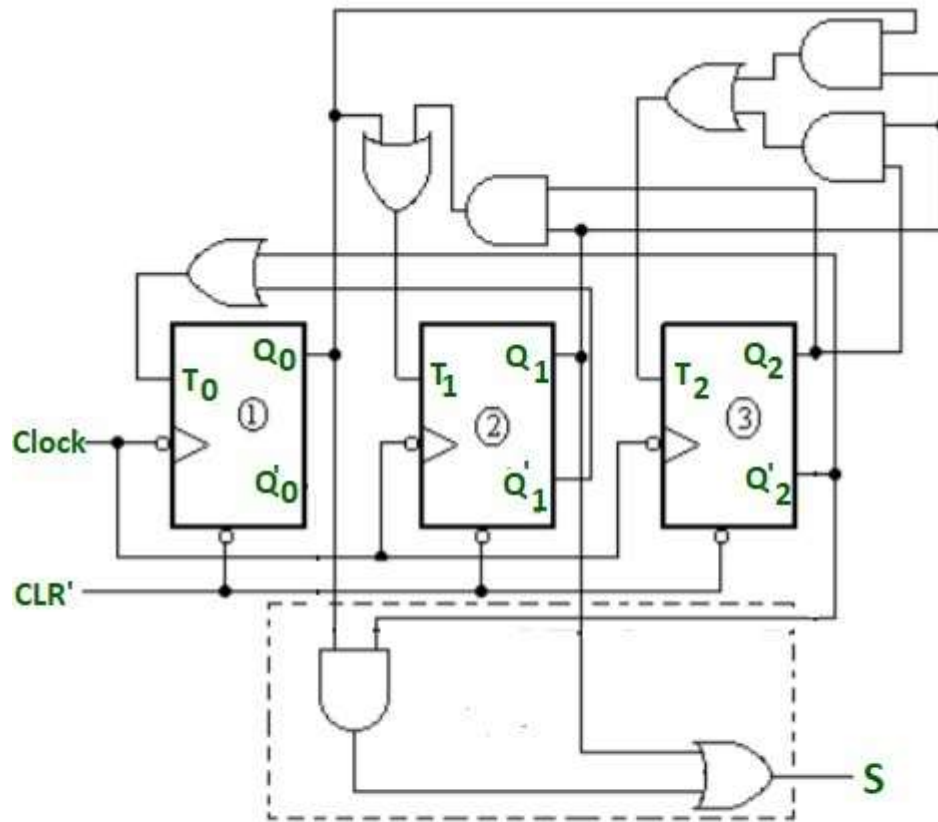
Q_2	Q_1	Q_0	S
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1

		$Q_1 Q_0$			
		00	01	11	10
Q_2	0	0	1	1	1
	1	0	0	X	1

$$S = Q_1 + Q_2' Q_0$$

Simplified expression for decoder circuit is $S = Q_1 + Q_2' Q_0$.

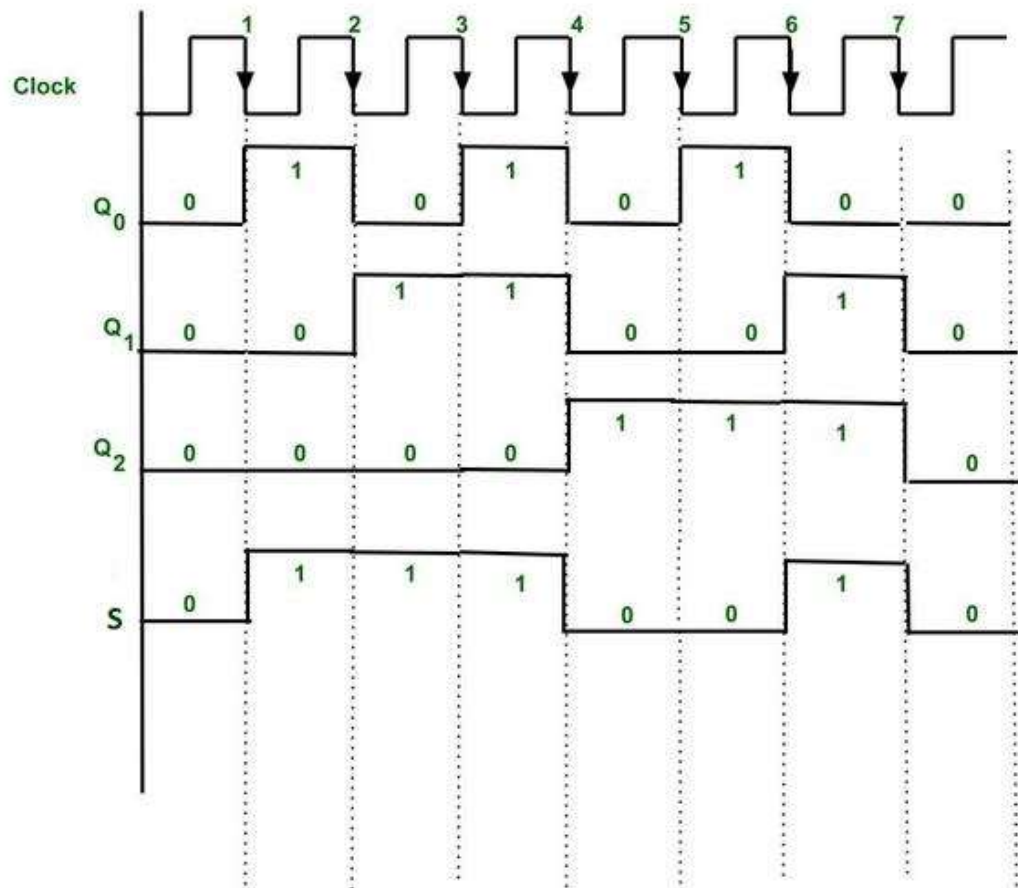
Example — When we are in state $Q_2 = 0$ $Q_1 = 0$ $Q_0 = 0$, then the value of $S = 0 + 0.1 = 0$.



Complete logic diagram with Decoder.

The decoder circuit generates output after every clock(-ve edge triggered) pulse. The output i.e S will be act as Control signal for other circuitry.

The generation of a control signal takes place after each negative edge clock and timing diagram for such control signal may also be drawn as shown below. Each counter state is used for generating one control signal.



Timing diagram for Control signal S generation by counter

2.3 Describe the status signal (S0 and S1) for Memory and I/O operation.

S0 and S1 status signals, along with IO/M, define the microprocessor's current operation (like fetch, read, write) and differentiate between memory and I/O access, with specific combinations indicating actions such as **Opcode Fetch (S1=1, S0=1, IO/M=0)**, **Memory Read (S1=1, S0=0, IO/M=0)**, **Memory Write (S1=0, S0=1, IO/M=0)**, **I/O Read (S1=1, S0=0, IO/M=1)**, **I/O Write (S1=0, S1=1, IO/M=1)**, and **Halt (S1=0, S0=0, IO/M=0)**, helping external devices understand what the processor needs.

Here's a breakdown of the S0 and S1 signals for memory and I/O operations (using the 8085 microprocessor as a standard example):

- **IO/M (Input/Output Memory):**
 - **Low (0):** Indicates a Memory operation (address bus holds a memory address).
 - **High (1):** Indicates an I/O operation (address bus holds an I/O port address).
- **S1 & S0 (Data Bus Status):** These signals, in combination with IO/M, define the exact machine cycle:

IO/M	S1	S0	Operation Type	Description
0	1	1	Opcode Fetch	CPU fetches instruction from memory.
0	1	0	Memory Read	CPU reads data from memory.
0	0	1	Memory Write	CPU writes data to memory.
1	1	0	I/O Read	CPU reads data from an I/O port.
1	0	1	I/O Write	CPU writes data to an I/O port.
0	0	0	Halt	CPU is halted.

In essence: The S0 and S1 signals provide a binary code that, when interpreted with the IO/M signal, tells other system components (like memory chips or I/O controllers) whether the processor is fetching an instruction, reading data, writing data, and whether that access is to memory or an I/O device.

2.4 Explain the function of the ALE and IO/M signal of the 8085 microprocessor.

This pin of the 8085 microprocessor, ALE, is one of the most important pins used in interfacing with external memory devices. During any machine cycle, while interfacing with an external memory using the 8085 microprocessor, the ALE pin latches the lower-order address bits, A0-A7.

During the beginning of each machine cycle, the microprocessor outputs the low-order address bits, A0-A7, onto the address bus. The ALE signal is sent to latch the valid address on the bus; the external devices attached to the microprocessor latch the address on the falling edge of the ALE.

Why ALE Pin is Required in the 8085 MicroProcessor?

The ALE pin is employed for the right synchronization of microprocessor address and data signals with those of external memory gadgets. It facilitates the proper interface of the microprocessor with external memory by providing a clear indication of when the address information is stable and can be used by external devices.

The other key pin on the 8085 microprocessor is the ALE pin, which makes efficient communication between the 8085 microprocessor and the memory devices sitting external to it to carry on all the memory access operations hassle-free.

The 8085 [microprocessor](#) uses the ALE pin to pull the slot at a low during any machine operations it takes while connecting with other computer memories; because it operates by latching the bottom addresses of eight bits (A0-A7) which are output from it. It is for the

purpose of maintaining a good level (or point) of control (or flow) within the computer such that there is consistency between what appears in terms of data as well as where things get stored for reference purposes when using external storage places.

Types of Classification

There are two main types of ALE pins in a microprocessor which depend on how it works.

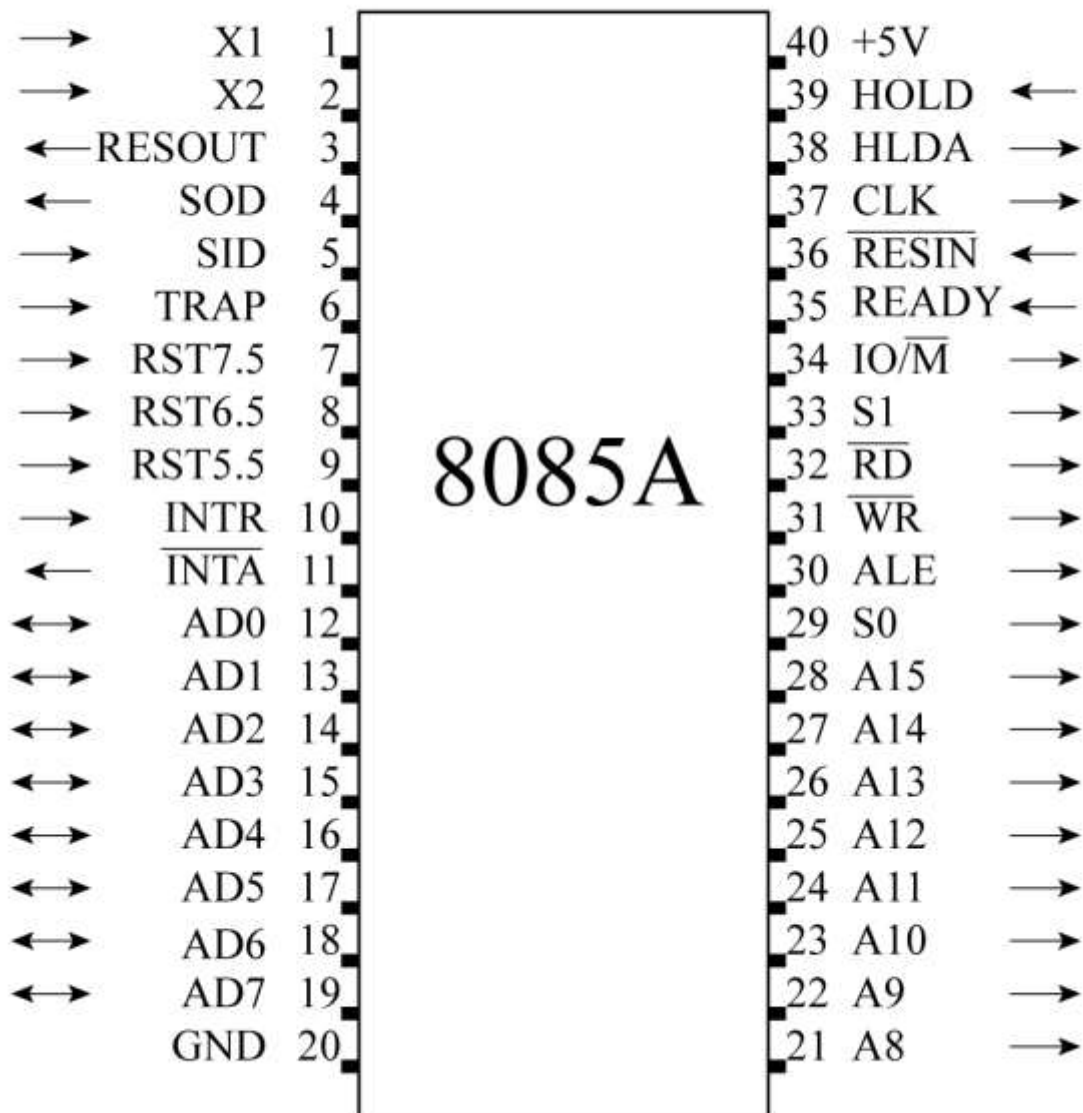
1. ALE Pin Active Low: One is active-high and has been identified as; Here it means that the ALE signal is active at high voltage (usually logic 1). This type contains an indication showing whether the addresses on the bus are good enough for latching by external devices.

2. ALE Pin High Active: In this type, active or valid is considered the ALE signal when it is at a high voltage level usually logic 1. Whenever the ALE pin is high, it means that the address on the bus is irrelevant and cannot be locked by external devices.

These classifications; high-active and low-active help in understanding how ALE pin behaves as regards signaling when address information on the bus should be treated as valid.

Diagram of Latching Circuit with Explanation

Pin Diagram of 8085



Pin Diagram of 8085

We know that AD0 to AD7 lines are multiplexed and the lower half of the address, A0-A7, is available only during T1 of the machine cycle. Now, this lower half of the address is needed during T2 and T3 of the machine cycle also, in order to access a particular location in memory or I/O port. It means that the lower half of an address bus has to be latched in T1 of the machine cycle, so that it is available throughout the machine cycle. To latch falling edge of the address, the 8085 microprocessor uses an external latch with the help of ALE (Address Latch Enable) signal. The figure shows the hardware connection for the latching the lower half of an address. IC 74LS373 contains an 8-bit latch and hence contains 8 D flip flops. Input gets transferred to the output only during clock high. The clock signal gets driven by ALE signal from 8085. The ALE signal is activated only during T1, so input will get transferred to

the output only during T1, i.e. address (A0- A7) on the AD0-AD7 multiplexed bus. In the remaining part of the machine cycle ALE signal is disabled so the output of the latch (A0-A7) remains unchanged. To latch lower half of an address, in each machine cycle, the 8085 gives ALE signal high during T1 of every machine cycle.

The Intel 8085 is an 8-bit microprocessor with 16 address lines designed to give a 16-bit memory location. Out of these 16 address lines, 8 higher order address bits are transmitted on separate eight bit lines and the remaining lower order is sent on another separate eight lines that are multiplexed with the 8 bit data lines. As an Address Enable Latch, ALE becomes a signal that is generated through a positive-going pulse every time there is a new operation initiated by a computer. When pulse goes low, meaning ALE=0, it indicates that the data bus is enabled; and when ALE=1, high pulse means the address bus is enabled.

Working Principle of ALE

A very important part of interfacing with external memory devices on the 8085 microprocessor is played by The address latch enable (ALE) pin. In order to send the lower 8 bits of the address to the memory or an I/O device, the microprocessor activates the ALE pin.

- The lower 8 bits of the memory address are put on the address bus by the microprocessor.
- Next, it turns on ALE. This means that the address on the bus is OK and must be latched by external devices.
- External latching circuits tied to the ALE pin then latch the address on the address bus.
- After the microprocessor takes over address latch, it uses the data bus to transfer data to or from this specific memory location.

The ALE pin of the 8085 microprocessor pin perform specific role in interfacing with external memory devices. For sending lower 8 bits of the address to the memory or I/O device, it activates the ALE pin.

Components of ALE

ALE as a term in the context of the 8085 microprocessor stands for the Address Latch Enable signal (8085). These are some of them on 8085 microprocessor board specific components pertaining ALE

1. Address Bus: An address bus is responsible for carrying memory-address from a microcontroller Unit to other components that include memories and input/output devices.

2. Data Bus: The data bus connects the microprocessor to main memory or I/O peripherals carrying information used in those areas--addresses with instructions or data.

3. Control Signals: Data flow and operations of the microprocessor are governed by various signals referred to as control signals, for example, RD (Read), WR (Write), and ALE (Address Latch Enable).

4. Address Latch: In case ALE signal is active, the address latch becomes a place where addresses from the address bus are captured before being used by external devices.

5. External Memory Interface: The microprocessor has an external memory interface that allows it to interact with external memory devices for data saving and retrieval purposes.

6. Input/output ports: These act like communication channels that facilitate exchange of data between microprocessor and all peripheral devices connected to it.

ALE one of pins instead latches addresses on an external address bus during each machine cycle consequently allowing them (external devices)to know what location should be available next time they want data from it: Arduino.

Construction:

The ALE (Address Latch Enable) pin is an output in the 8085 microprocessor. It has to generate a pulse when the microprocessor intends to latch the lower 8 bits of the memory address.

Physically, the ALE pin connects externally with latch circuits that in turn capture the address applied to the address bus. As part of the address strobe, when the microprocessor activates the ALE pin, a signal is sent out to the external latch circuits connected to the ALE pin, indicating the address on the bus has gone stable and is ready to be latched. Construction of ALE pin: Ensuring that it can generate a reliable pulse to trigger latching at address by external circuits, this functionality is important for proper communication between the microprocessor and external memory or I/O devices.

Important Terminologies

The 8085 microprocessor's ALE (Address Latch Enable) pin can refer to as follows.

ALE Pulse: When an ALE goes from 0-Vcc, it means the lowest eight bits in the memory's address are now validly placed on the address bus where it would subsequently be locked-on inside other electronics facilities.

Address Bus: The group of cables transmitting memory address information between microprocessor & external memory or I/O devices is called Address Bus. Enabling latching of the address, the ALE pin interacts with address bus.

External Latch Circuits: External latch circuits are linked to the ALE pin and are charged with grabbing and grasping onto the last 8 bits of the memory address when ALE makes a pulse.

Address Latching: Address latching describes the way external latch circuits hold onto the bottom eight bits of an address each time a pulse is sent through an external latch on its address enable input (ALE).

Knowing these words makes understanding the ALE pin process by which the information between the microprocessor and external devices about moving an address goes into focus.

Examples

One of the most important signals for the 8085 microprocessor is the Address Latch Enable (ALE) pin. For instance, let's examine how T-States explain the functionality of the ALE pin in relation to microprocessor operation timing.

Imagine a situation in which the 8085 microprocessor is going through one instruction level during which it should work with pin ALE. A signal ALE is produced within a machine cycle so as to latch lower address bits on to an external latch.

This is what the T states structure looks like in simpler terms for the ALE pin operation:

T1 : At the start of each machine cycle, the ALE signal goes high which shows that the microprocessor is almost ready to send out an address into the outside programs circuitry.

T2 : Lower bits of addresses stabilize onto address bus throughout this T-state to help ensure that an appropriate address is available to peripheral equipment.

T3 : Keeping the ALE signal active helps in grabbing the external latch for the lower address byte and maintains it till the end of the machine cycle.

This T states architecture structure of ALE pin operation assists the 8085 microprocessor in aligning the address information hence enabling a proper communication with external memory or devices.

Differences

Its specific function in relation to address latching is what sets the ALE pin apart as regards the other microprocessor pins.

Control signals, data transfer, or power supply may be seen in any other pin except ALE which is there to help in latching lower eight bits of memory address; as such it acts as a bridge between the internal bus system from one side and external memory from another side while enhancing the speed of data transfer simultaneously. This is when it sends out a pulse to show that the address is now stable on the address bus making it possible for external latch circuits to capture it for storage purposes.

It's important to understand that the ALE pin plays a unique role in enabling communication of memory addresses between microprocessor and external devices.

Advantages of ALE Pin

But the ALE pin of microprocessor has several advantages in the context of address handling and interfacing with external devices :

- **Improved Timing Control** : A pulse is produced at proper timing by the ALE pin to latch the lower 8 bits of the memory address, hence providing improved timing control that makes sure the external latch circuits capture the address information accurately.
- **Simplified Interface Design** : The microprocessor can easily communicate the lower 8 bits of the address to the external devices using the ALE pin itself, avoiding complex address decoding circuits. This simplifies the interface design between the microprocessor and external memory or i/o devices.
- **ALE allows efficient use of the address bus** : The ALE pin is a facility to indicate that the lower order address bits can be latched, that is, it is informing that this is stable. It thus enhances overall performance for efficient utilization of the address bus.
- **Flexibility in System Configuration** : With the availability of the ALE pin, a great deal of flexibility in system configuration is afforded by possibly latching the address information by external devices whenever required. This flexibility is very useful while interfacing many peripherals or modules of memory.
- **Reliability** : The presence of an ALE pin helps in reliable communication of the address information from the microprocessor to the external component, thus reducing the possibility of data errors or corruption during address transmission.

These advantages underpin the central role that the ALE pin plays in issues concerning handling, and then to the general efficiency and reliability of the microprocessor system.

Disadvantages of ALE Pin

Even though a microprocessor's ALE pin has three primary advantages when dealing with addresses, this does not happen sans associated limitations or disadvantages:

- **Complexity in Circuitry**: The addition of an ALE pin to capture and hold the bits at the lower address requires additional external latch circuits. Thus, this structure increases the system design complexity and the count of components.
- **Timing Constraints**: The ALE signal should be precisely timed to latch address information; otherwise, it will not correctly latch. The occurrence of any type of timing fault or delay in generating an ALE pulse may result in data integrity issues or even address decoding errors.
- **Potential for Signal Glitches**: When the ALE signal does not align perfectly with the address bus signals, this can result in signal glitches during address transitions and noise. This may ultimately affect the reliability of transfers conveying information about addresses.

- **Limited Addressing Range:** Typically, the ALE pin latches only the lower 8 bits of the memory address. Lots of other mechanisms or signals may be needed, especially in a large address space, to handle upper address bits—making addressing schemes more complex.
- **Dependency on External Components:** The operation of the ALE pin is dependent on the external latch circuits to capture the address information. Failures or malfunctioning in parts connected externally, like these, may affect the latching of an address properly.

Understanding this disadvantage should be able to help weigh the trade-offs associated with the use of the ALE pin in microprocessor systems.

Applications of ALE

The ALE pin of a microprocessor finds applications in several other scenarios where precise address handling and interfacing with other external devices are required. The common applications of the ALE pin include :

- **Memory Interfacing:** Since the microprocessor has to access external memory devices in some systems, the ALE pin is used to latch the lower address bits for selecting the memory location. This would help in correct addressing of memory locations and quick retrieval of data from them.
- **I/O Device Interface:** The ALE pin permits latching of the address information for accessing certain registers or ports installed on the external devices when input-output devices are interfaced to the microprocessor. It will also permit undisturbed communication between the microprocessor and peripherals.
- **Multiplexed Address Bus:** In some microprocessors, address and data share the same lines, and the ALE pin is used to de-multiplex the address and data. This means that the address information is separated through this pin so that it can be understood by external devices.
- **DMA:** In which this ALE pin is used to transmit the memory address information precisely to the [DMA controller](#) while performing a direct data transfer from peripherals into the memory and vice versa by means of DMA controllers, that are outside the intervention of any CPU within the process.
- **System Control Signals:** The ALE pin can also be used to generate control signals for various system operations based on the address information. For instance, it can trigger some action or functionality of the system depending on the accessed memory address.

2.5 Explain the need to De-multiplex the bus AD7- AD0.

Demultiplexing the [AD7-AD0 bus](#) in an 8085 microprocessor is crucial because these lines have a dual role: carrying the lower half of the 16-bit address and then acting as the 8-bit data bus, requiring an external latch (like the 74LS373) to capture the address during the first clock cycle so it isn't lost and can be sent to memory/I/O devices for correct decoding, optimizing pin usage by sharing lines.

Why Demultiplexing is Necessary

- **Pin Efficiency:** The 8085 has a 16-bit address bus but only 8 pins (AD0-AD7) shared with the data bus to save pins.
- **Address vs. Data:** During the first part of a machine cycle, AD0-AD7 carry the lower-order address bits (A0-A7).
- **Address Latching:** These address bits are only present for one clock cycle before the bus switches to data transfer.
- **External Latch:** An external latch (e.g., 74LS373) controlled by the [Address Latch Enable](#) (ALE) signal captures these low-order address bits.
- **Full Address Access:** The latched low-order address (A0-A7) combines with the high-order address (A8-A15, which stay on the bus longer) to form the complete 16-bit address needed by memory or I/O devices.
- **Correct Interfacing:** Without demultiplexing, the address would mix with data, causing memory/I/O to read the wrong information.

How It Works (Simplified)

1. **Address Cycle:** The microprocessor puts A0-A7 on AD0-AD7 and asserts the ALE signal high.
2. **Latch Action:** The latch takes a snapshot (latches) A0-A7.
3. **Data Cycle:** ALE goes low, freeing AD0-AD7 to carry data, while the latch holds the original address.
4. **Device Access:** Memory/I/O uses the latched A0-A7 and the stable A8-A15 to access the correct location.

2.6 If the clock frequency is 5Mhz, how much time is required to execute an instruction of 18 T-States?

Clock frequency = 5 MHz

So one clock cycle (T) = $1 / 5,000,000$ seconds

= 0.2 microseconds

= 200 nanoseconds

Instruction needs **18 states** (aka 18 clock cycles).

Time required = $18 \times 0.2 \mu\text{s}$
= **3.6 microseconds**

2.7 Explain the structure of RAM and ROM.

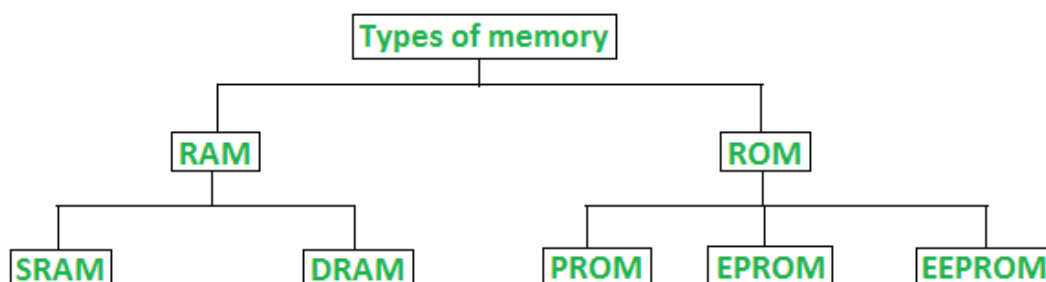
Memory is a fundamental component of computing systems, essential for performing various tasks efficiently. It plays a crucial role in how computers operate, influencing speed, performance, and data management. In the realm of computer memory, two primary types stand out: Random Access Memory (RAM) and Read-Only Memory (ROM). Understanding the difference between RAM and ROM is vital for appreciating their respective roles in a computer's operation. While RAM enables efficient processing and multitasking, ROM provides the necessary stability and reliability for system functions. Together, these memory types ensure that computers can operate smoothly and effectively in a digital landscape.

Types of Memory

Memory is the most essential element of a computing system because without it computer can't perform simple tasks. Both types of memory (RAM and ROM) are important for the computer, but they serve different purposes. RAM is used to store data that the computer is currently using, while ROM is used to store data that the computer needs to boot and operate. RAM is faster than ROM, as the data stored in it can be accessed and modified in any order, while data stored in ROM can only be read.

Computer memory is of two basic types:

1. Primary memory (RAM and ROM)
2. Secondary memory (Hard Drive, CD, etc).



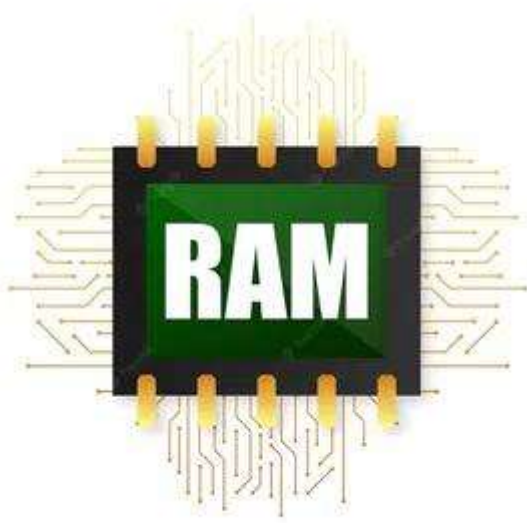
Classification of computer memory

Classification of Primary Memory

Random Access Memory (RAM)

Random Access Memory (RAM) is a type of computer memory that is used to temporarily store data that the computer is currently using or processing. RAM is volatile memory, which means that the data stored in it is lost when the power is turned off. RAM is typically used to store the operating system, application programs, and data that the computer is currently using.

- It is also called read-write *memory* or the *main memory* or the *primary memory* .
- The programs and data that the CPU requires during the execution of a program are stored in this memory.
- It is a volatile memory as the data is lost when the power is turned off.



Random Access Memory

Types of Random Access Memory (RAM)

- [Static RAM \(SRAM\)](#)
- [Dynamic RAM \(DRAM\)](#)

1. Static RAM: SRAM stands for Static Random Access Memory. It is a type of semiconductor which is widely used in computing devices and microprocessors.

2. Dynamic RAM: DRAM stands for Dynamic Random Access Memory. It is made of Capacitors and has smaller data life span than Static RAM.

Difference Between DRAM and SRAM

Parameter	DRAM	SRAM
Construction	Constructed of tiny capacitors that leak electricity.	Constructed of circuits similar to D flip-flops.

Data Retention	Requires a recharge every few milliseconds to maintain its data.	Holds its contents as long as power is available.
Cost	Inexpensive.	Expensive.
Speed	Slower than SRAM.	Faster than DRAM.
Storage Capacity	Can store many bits per chip.	Can not store many bits per chip.
Power Consumption	Uses less power.	Uses more power.
Heat Generation	Generates less heat.	Generates more heat.
Typical Usage	Used for main memory.	Used for cache.

Advantages of Random Access Memory (RAM)

- **Speed:** RAM is much faster than other types of storage, such as a hard drive or solid-state drive, which means that the computer can access the data stored in RAM more quickly.
- **Flexibility:** RAM is volatile memory, which means that the data stored in it can be easily modified or deleted. This makes it ideal for storing data that the computer is currently using or processing.
- **Capacity:** The capacity of RAM can be easily upgraded, which allows the computer to store more data in memory and thus improve performance.
- **Power Management:** RAM consumes less power compared to hard drives, and solid-state drives, which makes it an ideal memory for portable devices.

Disadvantages of Random Access Memory (RAM)

- **Volatility:** RAM is volatile memory, which means that the data stored in it is lost when the power is turned off. This can be a problem for important data that needs to be preserved, such as unsaved work or files that have not been backed up.
- **Capacity:** The capacity of RAM is limited, and although it can be upgraded, it may still not be sufficient for certain applications or tasks that require a lot of memory.
- **Cost:** RAM can be relatively expensive compared to other types of memory, such as hard drives or solid-state drives, which can make upgrading the memory of a computer or device more costly.

Read-Only Memory (ROM)

Read Only Memory (ROM) is a type of computer memory that is used to permanently store data that does not need to be modified. ROM is non-volatile memory, which means that the

data stored in it is retained even when the power is turned off. ROM is typically used to store the computer's BIOS (basic input/output system), which contains the instructions for booting the computer, as well as firmware for other hardware devices.

- Stores crucial information essential to operate the system, like the program essential to boot the computer.
- It is non-volatile.
- Always retains its data.
- Used in embedded systems or where the programming needs no change.
- Used in calculators and peripheral devices.
- ROM is further classified into four types- M ROM , [PROM](#) , [EPROM](#) , and [EEPROM](#) .



Read-Only Memory (ROM)

Types of Read-Only Memory (ROM)

1. [PROM \(Programmable Read-Only Memory\)](#)
2. [EPROM \(Erasable Programmable Read Only Memory\)](#)
3. [EEPROM \(Electrically Erasable Programmable Read Only Memory\)](#)
4. MROM (Mask Read Only Memory)

1. PROM (Programmable read-only memory): It can be programmed by the user. Once programmed, the data and instructions in it cannot be changed.

2. EPROM (Erasable Programmable read-only memory): It can be reprogrammed. To erase data from it, expose it to ultraviolet light. To reprogram it, erase all the previous data.

3. EEPROM (Electrically erasable programmable read-only memory): The data can be erased by applying an electric field, with no need for ultraviolet light. We can erase only portions of the chip.

4. MROM(Mask ROM): Mask ROM is a kind of read-only memory, that is masked off at the time of production. Like other types of ROM, mask ROM cannot enable the user to change the data stored in it. If it can, the process would be difficult or slow.

Advantages of Read Only Memory (ROM)

- **Non-volatility:** ROM is non-volatile memory, which means that the data stored in it is retained even when the power is turned off. This makes it ideal for storing data that does not need to be modified, such as the BIOS or firmware for other hardware devices.
- **Reliability:** Because the data stored in ROM is not easily modified, it is less prone to corruption or errors than other types of memory.
- **Power Management:** ROM consumes less power compared to other types of memory, which makes it an ideal memory for portable devices.

Disadvantages of Read Only Memory (ROM)

- **Limited Flexibility:** ROM is read-only memory, which means that the data stored in it cannot be modified. This can be a problem for applications or firmware that need to be updated or modified.
- **Limited Capacity:** The capacity of ROM is typically limited, and upgrading it can be difficult or expensive.
- **Cost:** ROM can be relatively expensive compared to other types of memory, such as hard drives or solid-state drives, which can make upgrading the memory of a computer or device more costly.

Difference Between RAM and ROM

Parameter	RAM	ROM
Storage Type	Temporary Storage.	Permanent Storage.
Storage Capacity	Store data in MBs.	Store data in GBs.
Data Volatility	Volatile.	Non-volatile.
Usage	Used in normal operations.	Used for startup process of computer.
Data Writing Speed	Writing data is faster.	Writing data is slower.

2.8/2.9 Explain Memory Read-Write operation.

A memory unit stores binary information in groups of bits called words. Data input lines provide the information to be stored into the memory, Data output lines carry the

information out from the memory. The control lines Read and write specifies the direction of transfer of data. Basically, in the memory organization, there

are 2^l memory locations indexing from 0 to $2^l - 1$ where l is the address buses. We can describe the memory in terms of the bytes using the following formula: $N = 2^l \text{ Bytes}$ Where, l is the total address buses N is the memory in bytes For example, some storage can be described below in terms of bytes using the above formula:

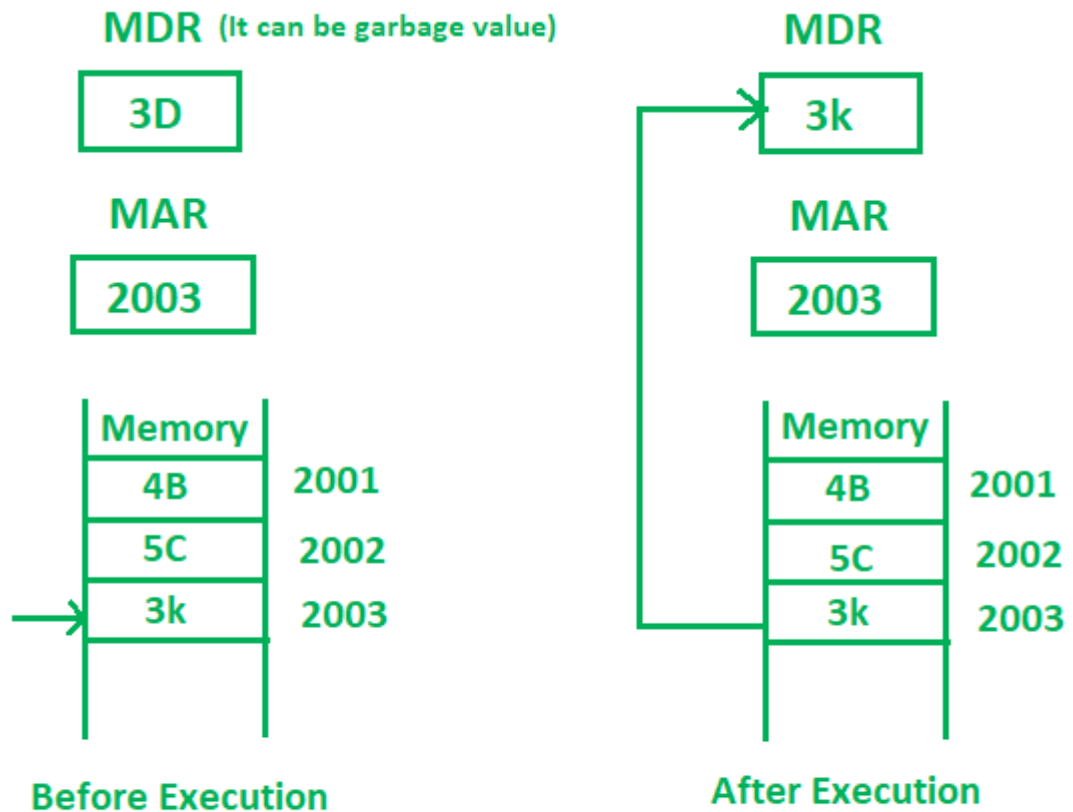
$$1\text{kB} = 2^{10} \text{ Bytes}$$

$$64 \text{ kB} = 2^6 \times 2^{10} \text{ Bytes} \\ = 2^{16} \text{ Bytes}$$

$$4 \text{ GB} = 2^2 \times 2^{10}(\text{kB}) \times 2^{10}(\text{MB}) \times 2^{10}(\text{GB}) \\ = 2^{32} \text{ Bytes}$$

Memory Address Register (MAR) is the address register which is used to store the address of the memory location where the operation is being performed. **Memory Data Register (MDR)** is the data register which is used to store the data on which the operation is being performed.

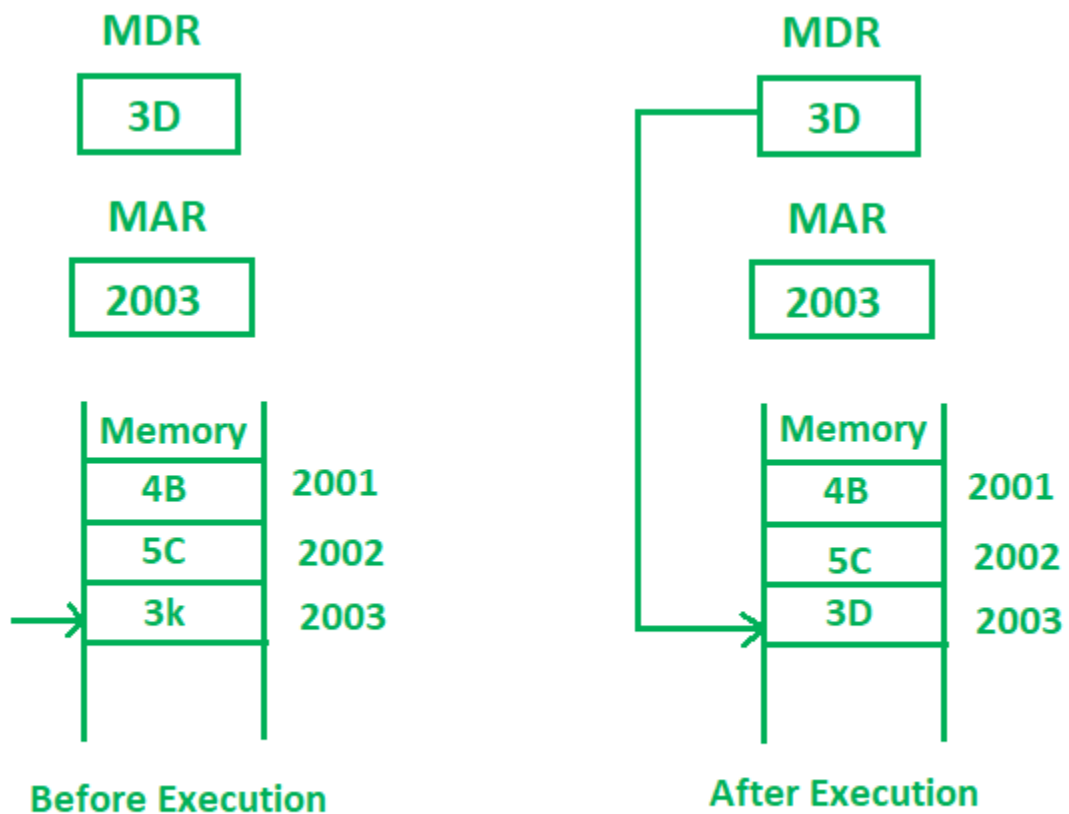
1. **Memory Read Operation:** Memory read operation transfers the desired word to address lines and activates the read control line. Description of memory read operation is given below:



Memory Read Operation

In the above diagram initially, MDR can contain any garbage value and MAR is containing 2003 memory address. After the execution of read instruction, the data of memory location 2003 will be read and the MDR will get updated by the value of the 2003 memory location (3D).

2. **Memory Write Operation:** Memory write operation transfers the address of the desired word to the address lines, transfers the data bits to be stored in memory to the data input lines. Then it activates the write control line. Description of the write operation is given below:



Memory Write Operation

In the above diagram, the MAR contains 2003 and MDR contains 3D. After the execution of write instruction 3D will be written at 2003 memory location.

Advantages of Read Operations:

Speed: Read operations are generally faster than write operations since the data is already stored in the memory.

Efficiency: Read operations are more efficient since they do not require modifying the data in memory.

Non-destructive: Read operations do not modify the data in memory, so they can be performed repeatedly without affecting the stored data.

Disadvantages of Read Operations:

Limited functionality: Read operations only retrieve data from memory, so they cannot be used to modify the data.

Security risks: Read operations can be used to access sensitive data stored in memory, making them a potential security risk.

Advantages of Write Operations:

Flexibility: Write operations allow data to be modified, making them useful for storing and updating information in memory.

Dynamic: Write operations allow data to be changed in real-time, making them essential for many computing applications.

Customization: Write operations allow users to customize and personalize their computing experience by modifying stored data.

Disadvantages of Write Operations:

Slower: Write operations are generally slower than read operations since the data needs to be modified and then written back to memory.

Overwriting risk: Write operations can overwrite existing data in memory, leading to data loss or corruption.

Wear and Tear: Repeated write operations can cause wear and tear on memory cells, leading to reduced reliability and lifespan.

2.10 Which is the instruction used for increment and decrement purpose? Give example of it.

Instruction used for Increment and Decrement

In a microprocessor, **increment and decrement operations** are performed using the **INC** and **DEC** instructions. These instructions are used to increase or decrease the contents of a register or a memory location by **one**. They are commonly used in counters, loops, and address manipulation.

INC (Increment Instruction):

The INC instruction increases the value of a register or memory location by 1. It affects all flags except the Carry flag.

Examples:

- INC A → Increments the contents of accumulator A by 1
- INC B → Increments register B by 1
- INC M → Increments the memory content pointed by HL pair

DEC (Decrement Instruction):

The DEC instruction decreases the value of a register or memory location by 1. It also affects all flags except the Carry flag.

Examples:

- DEC A → Decrements the contents of accumulator A by 1
- DEC C → Decrements register C by 1
- DEC M → Decrements the memory content pointed by HL pair

Q.3 Describe the following questions. (7-8 Marks).

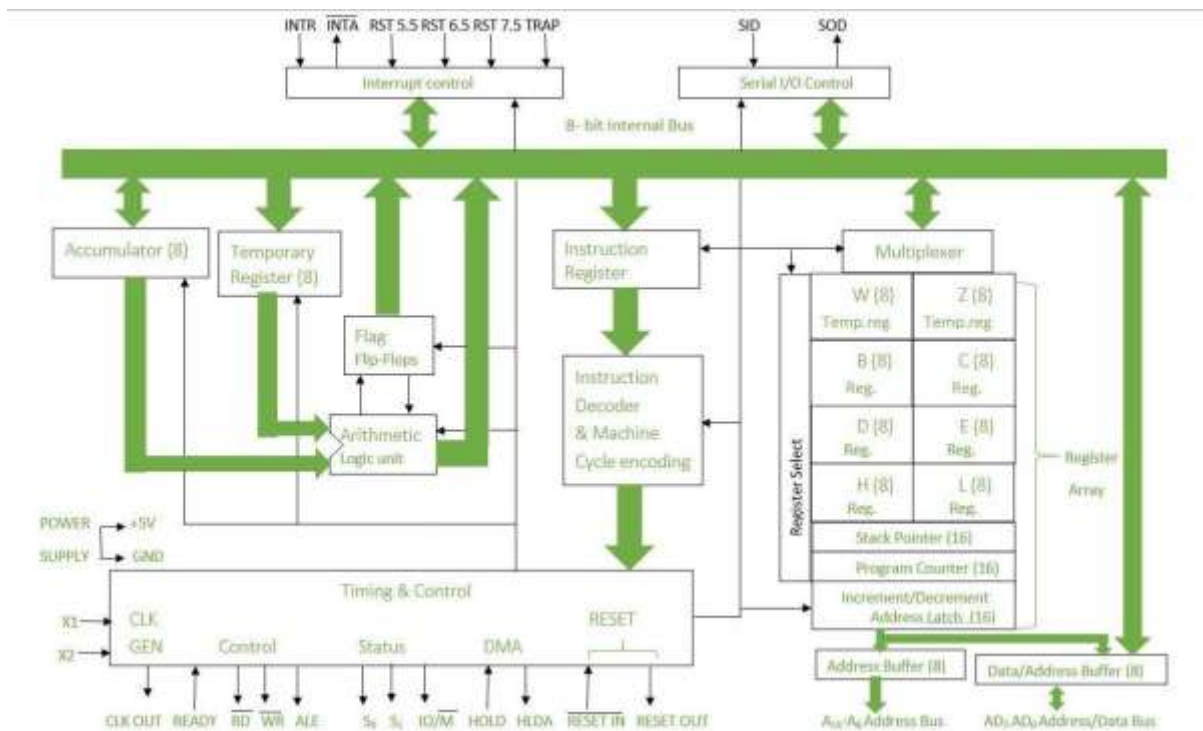
3.1 Draw the functional block diagram of internal architecture of IC 8085 and explain its working.

The 8085 Microprocessor's internal architecture features an **ALU**, [Timing & Control Unit](#), and [Registers](#), with registers broken into General Purpose (B, C, D, E, H, L), [Program Counter](#) (PC), [Stack Pointer](#) (SP), and [Flag Register](#). Its working involves fetching instructions via the multiplexed [Address/Data Bus](#), decoding them in the [Instruction Register & Decoder](#), executing operations (arithmetic/logic in the ALU) controlled by the Timing Unit, and managing data flow with latches and buses, all synchronized by the [internal clock](#).

Functional Block Diagram (Conceptual)

(Imagine a diagram with these main sections)

- [Arithmetic and Logic Unit \(ALU\)](#): Performs calculations (add, subtract) and logical operations (AND, OR, XOR).
- [Timing and Control Unit](#): Generates control signals (RD, WR, ALE, IO/M) to synchronize operations, fetching, and data transfer.
- **Registers:**
 - **Accumulator (A)**: 8-bit register for storing operands/results.
 - **General-Purpose Registers (B, C, D, E, H, L)**: 8-bit, used for data storage/manipulation.
 - **Program Counter (PC)**: 16-bit, points to the next instruction address.
 - **Stack Pointer (SP)**: 16-bit, points to the stack top.
 - **Instruction Register (IR)**: Holds the fetched instruction.
 - [Temporary Register](#): Holds data temporarily during ALU operations.
- [Interrupt Control](#): Handles external interrupt signals.
- [Serial I/O Control](#): Manages SID (Serial In) and SOD (Serial Out) pins.
- [Address/Data Bus Buffers & Latches](#): Manage the shared 8-bit multiplexed bus.



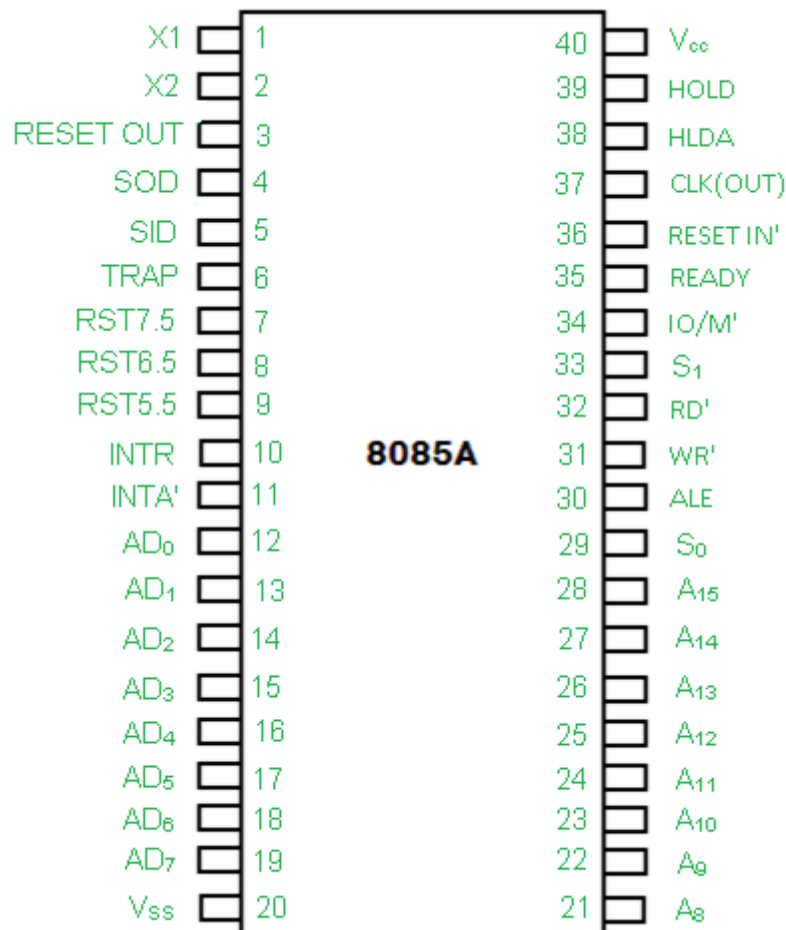
Working Principle

1. **Fetch:** The PC provides the address, sent via the multiplexed bus (AD0-AD7) as address bits (with ALE pulse) to memory.
2. **Instruction Decode:** The fetched instruction goes to the IR, then the Instruction Decoder interprets it.
3. **Execute:**
 1. The **Timing & Control Unit** generates signals (RD, WR) to fetch data from or write data to memory/I/O.
 2. Data moves to the ALU via registers.
 3. The ALU performs the operation (e.g., addition), storing the result in the Accumulator and setting **Flags** (Zero, Carry, Sign, etc.).
 4. **Data Transfer/I/O:** Data is moved between registers, memory, or I/O devices using the address/data bus.
 5. **Control:** The built-in clock synchronizes all internal operations.
 6. **Interrupts:** If an interrupt occurs, the CPU pauses execution, handles the interrupt routine (ISR), and resumes.

3.2 Draw the Pin diagram of 8085 Microprocessor and explain each pin.

The **8085 microprocessor** is a popular 8-bit microprocessor developed by Intel. It has 40 pins, each with a specific function for interfacing with memory, input/output devices, and other components.

Pin diagram of 8085 microprocessor is shown below:



Key Pin Descriptions

1. Address Bus and Data Bus

Address Bus (A8 to A15): The address bus is unidirectional, i.e., bits flow in one direction from the microprocessor unit to the peripheral devices and uses the higher order address bus.

Address Data Bus (AD0 to AD7): These are bi-directional data pins used to transfer data between the microprocessor and memory or I/O devices. The microprocessor is an 8-bit processor, so it uses 8 data lines. These pins serve the dual purpose of transmitting lower order address and data byte. During 1st clock cycle, these pins act as lower half of address. In remaining clock cycles, these pins act as **data bus**.

2. Control and Status Signals

ALE - It is an Address Latch Enable signal. It goes high during first T state of a machine cycle and enables the lower 8-bits of the address, if its value is 1 otherwise data bus is activated.

IO/M' - It is a status signal which determines whether the address is for input-output or memory. When it is high(1) the address on the address bus is for input-output devices. When it is low(0) the address on the address bus is for the memory.

SO, S1 - These are status signals. They distinguish the various types of operations such as halt, reading, instruction fetching or writing.

IO/M'	S1	S0	Data Bus Status
0	1	1	Opcode fetch
0	1	0	Memory read
0	0	1	Memory write
1	1	0	I/O read
1	0	1	I/O write
1	1	1	Interrupt acknowledge
0	0	0	Halt

RD' - It is a signal to control READ operation. When it is low the selected memory or input-output device is read.

WR' - It is a signal to control WRITE operation. When it goes low the data on the data bus is written into the selected memory or I/O location.

READY - It senses whether a peripheral is ready to transfer data or not. If READY is high(1) the peripheral is ready. If it is low(0) the microprocessor waits till it goes high. It is useful for interfacing low speed devices.

3. Power Supply and Clock Frequency

Vcc - +5v power supply

Vss - Ground Reference

X1, X2 - A crystal is connected at these two pins. The frequency is internally divided by two, therefore, to operate a system at 3MHZ the crystal should have frequency of 6MHZ.

CLK (OUT) - This signal can be used as the system clock for other devices.

4. Interrupts and Peripheral Initiated Signals:

The 8085 has five interrupt signals that can be used to interrupt a program execution.

- (i) INTR
- (ii) RST 7.5
- (iii) RST 6.5
- (iv) RST 5.5
- (v) TRAP

The microprocessor acknowledges Interrupt Request by INTA' signal. In addition to Interrupts, there are three externally initiated signals namely RESET, HOLD and READY. To respond to HOLD request, it has one signal called HLDA.

1. **INTR (Interrupt Request):** This pin is used to request an interrupt from an external device.
2. **RST7.5, RST6.5, RST5.5:** These are vectored interrupts with different priority levels.
3. **TRAP:** This is a non-maskable interrupt and has the highest priority.
4. **INTA (Interrupt Acknowledge):** This pin is used to acknowledge interrupts from external devices.

5. Reset Signals

RESET IN - When the signal on this pin is low(0), the program-counter is set to zero, the microprocessor unit is reset.

RESET OUT - This signal indicates that the MPU is being reset. The signal can be used to reset other devices.

6. DMA Signals

HOLD - It indicates that another device is requesting the use of the address and data bus. Having received HOLD request the microprocessor relinquishes the use of the buses as soon as the current machine cycle is completed. Internal processing may continue. After the removal of the HOLD signal the processor regains the bus.

HLDA - It is a signal which indicates that the hold request has been received after the removal of a HOLD request, the HLDA goes low.

7. Serial I/O Ports

Serial transmission in 8085 is implemented by the two signals.

SID and SOD - SID is a data line for serial input whereas SOD is a data line for serial output.

Features of 8085 Microprocessor

1. **8-bit Processor:** Processes 8 bits of data at a time.
2. **16-bit Address Bus:** Can address up to 64 KB of memory.
3. **5 MHz Clock Speed:** Provides a decent processing speed for basic tasks.
4. **Low Power Consumption:** Ideal for embedded systems with minimal power needs.
5. **Interrupt System:** Features 5 interrupt pins for external signal response.

Limitations of 8085 Microprocessor

1. **Limited Memory Addressing:** Can address only up to 64KB of memory due to its 16-bit address bus.
2. **No Multiprocessing Support:** Cannot connect multiple processors for parallel processing.
3. **Limited Instruction Set:** Supports only 50 instructions, restricting its functionality for complex tasks.
4. **No Direct Memory Access:** Lacks DMA (Direct Memory Access) support, which slows data transfer between memory and peripherals.
5. **Slower Processing:** With a clock speed of up to 5 MHz, it is slower compared to modern processors.

3.3 Explain Memory interfacing with necessary steps and diagram.

Introduction :

The 8051 microcontroller is a popular microcontroller used in various applications, ranging from small embedded systems to larger industrial applications. While the 8051 has a limited amount of internal memory, it is possible to extend the memory capacity by interfacing it with external memory devices.

External memory interfacing in 8051 microcontroller involves connecting external memory devices such as RAM and ROM to the microcontroller to provide additional memory space. This allows the microcontroller to execute larger and more complex programs, store more data, and perform more complex operations.

External memory interfacing typically involves connecting the memory devices to the microcontroller through a data bus and an address bus. The data bus is used to transfer data between the microcontroller and the memory device, while the address bus is used to select a specific memory location in the memory device.

To interface with external memory, the 8051 microcontroller uses dedicated pins such as ALE (Address Latch Enable), PSEN (Program Store Enable), and RD (Read) and WR (Write) signals.

These signals are used to control the flow of data between the microcontroller and the external memory device.

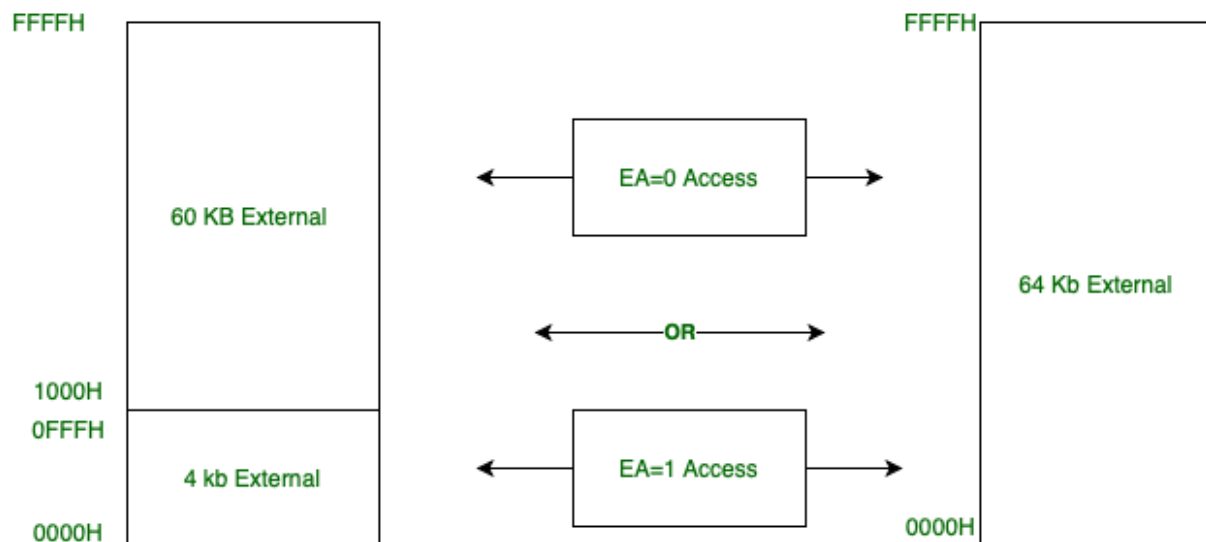
For minimal memory applications, the 8051 has internal data and code memory. In such a position. For certain applications, this memory capacity will not be adequate. To expand the memory space of the 8051 micro-controller, we must bind external ROM/EPROM and RAM. We also understand that ROM serves as program memory and RAM serves as data memory. Let's take a look at how the 8051 accesses these memories.

Why need External Memory Interfacing in 8051 Microcontroller ?

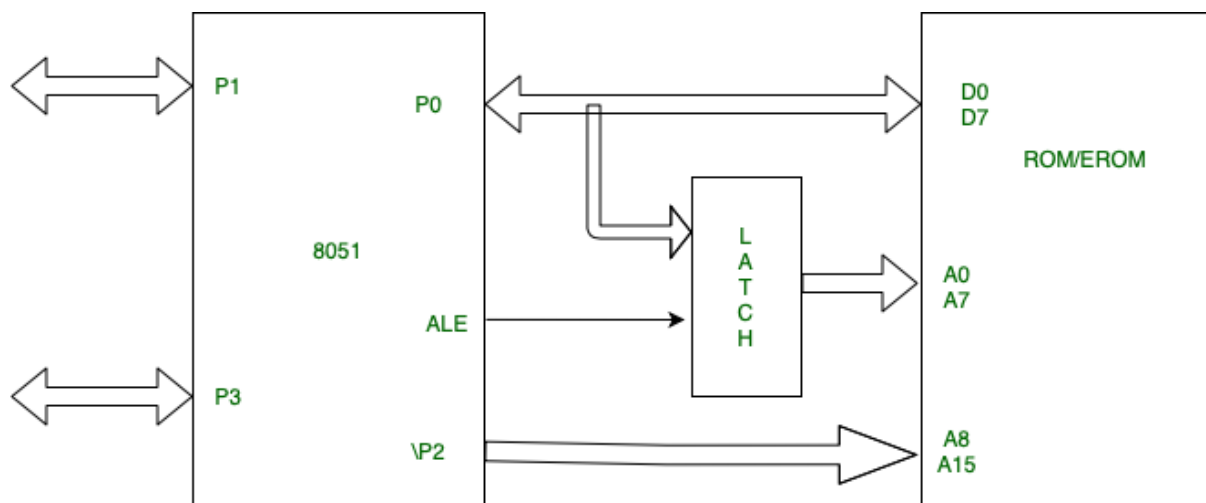
External memory interfacing is necessary in the 8051 microcontroller for several reasons:

1. **Limited internal memory:** The 8051 microcontroller has a limited amount of internal memory, including 128 bytes of RAM and 4KB of on-chip ROM. This memory may not be sufficient for some applications that require larger program memory or more data storage.
2. **Larger programs:** For applications that require larger programs, such as complex algorithms or multiple functions, external memory interfacing can provide the necessary program memory space to store these programs.
3. **Data storage:** Applications that require the storage of large amounts of data, such as data logging or data analysis, may require external memory interfacing to store the data.
4. **Flexibility:** External memory interfacing provides greater flexibility in the design of embedded systems, allowing for customization and adaptability to meet specific application requirements.
5. **Cost-effective:** External memory devices such as RAM and ROM are relatively inexpensive, making it cost-effective to interface them with the microcontroller instead of using more expensive on-chip memory.

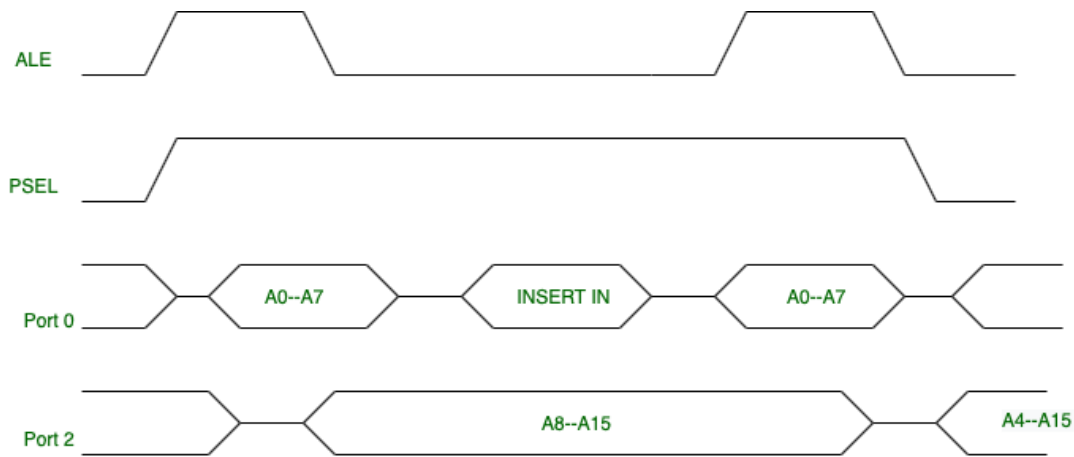
External Program Memory :



- The program fetches to addresses 0000H through 0FFFH are directed to the internal ROM in the 8051 when the EA pin is attached to Vec, and program fetches to addresses 1000H through FFFFH are directed to the external ROM/EPROM. When the EA pin is grounded, all addresses fetched by the program (0000H to FFFFH) are led to it.



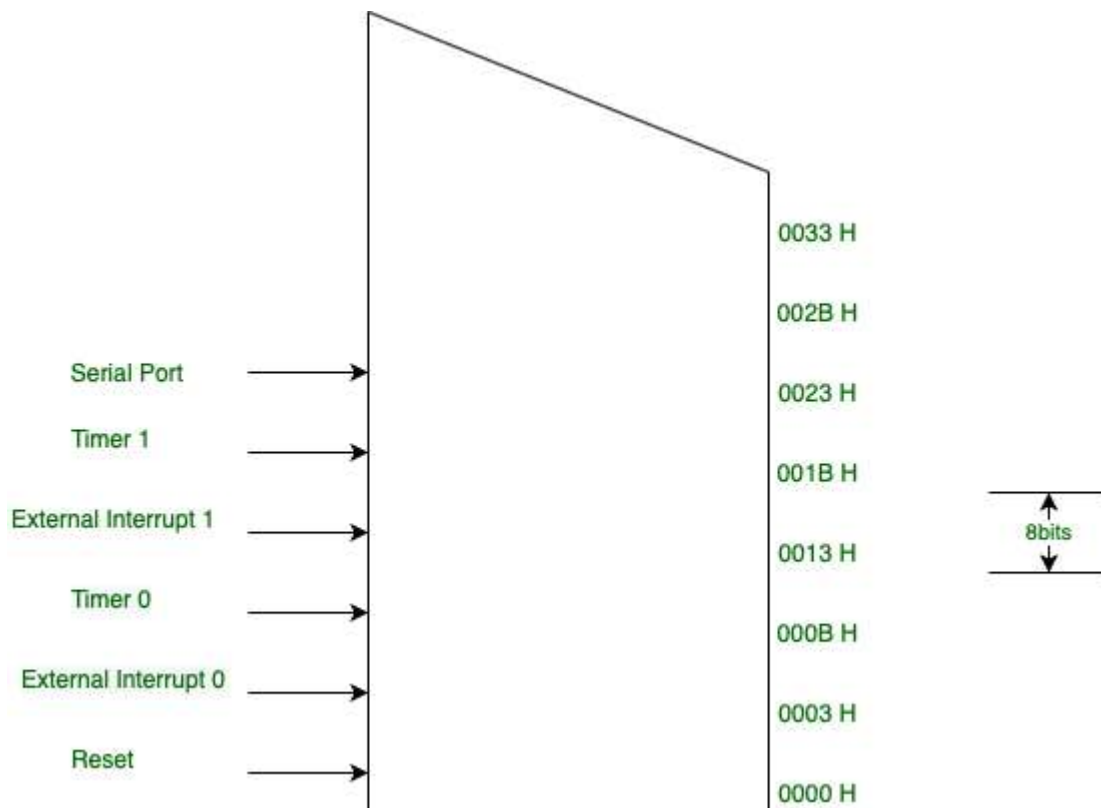
- ROM/EPROM that is external to the device. As seen in Fig. 1, the PSEN signal is used to trigger output e external ROM/EPROM.
- Port 0 is used as a multiplexed address/bus, as seen in Fig2. In the initial T-cycle, it provides a lower order 8-bit address, and later it is used as a data bus. The external latch and the ALE signal provided by the 8051 are used to latch the 8-bit address.



timing waveform for external data memory write cycle

Timing Waveform for external program memory

- Remote ROM/EPROM(Read Only Memory/Electronic Programmable ROM/Electronic Programmable ROM/Electronic Program The PSEN signal is used to activate output e external ROM/EPROM, as seen in Fig. 1.
- As seen in Fig2, port 0 is used as a multiplexed address/bus. It supplies a lower-order 8-bit address in the first T-cycle and later serves as a data bus. The 8-bit address is latched using the external latch and the ALE signal given by the 8051.



- 1External Interrupt 0 is 0003H, Timer 0 is 000BH, External Interrupt 1 is 0013H, Timer 1 is 001BH, and so on. If an interrupt is to be used, the operation routine for it must be in the same place as the interrupt. If the interrupt isn't used, the service location may be used as general-purpose program memory.

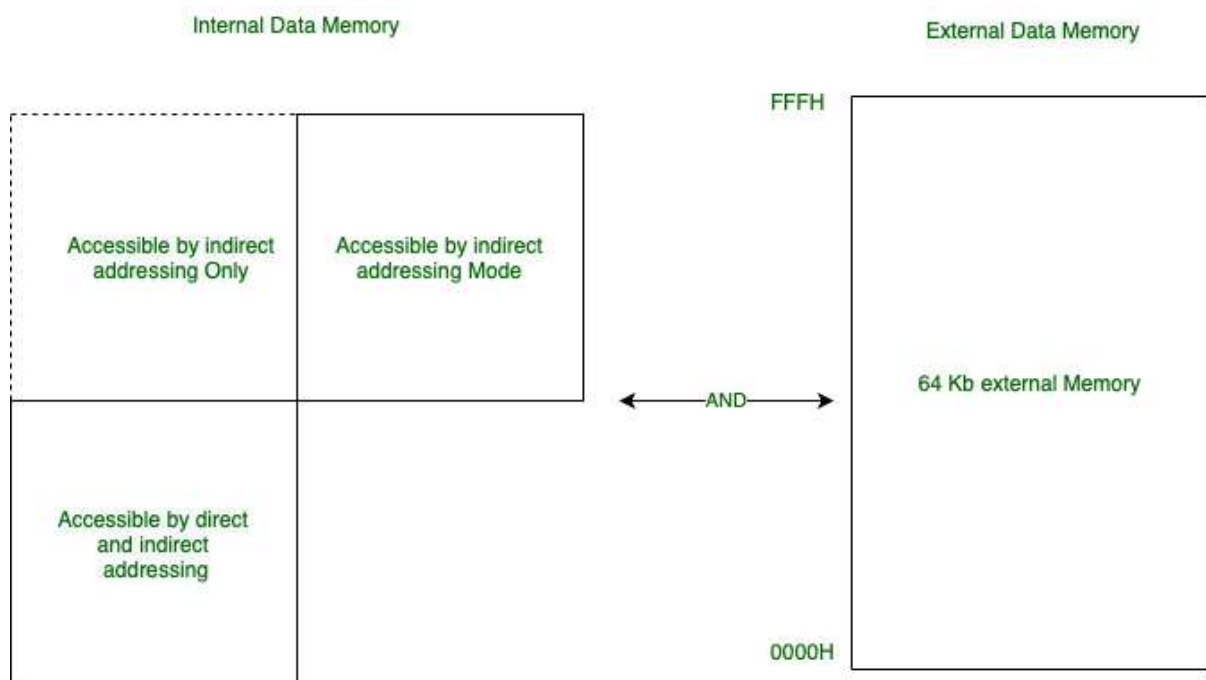
Instructions to Access External ROM / Program Memory :

This table is explaining the instructions to access external ROM/program memory.

Mnemonic	Operation
MOVC A, @ A+DPTR	Copy the contents of the external ROM address formed by adding A and the DPTR, to A
MOVC A, @ A + PC	This operation will do copy This operation contents of the external ROM address formed by adding A and the PC, to A.

External Memory Interfacing :

- Up to 64 k-bytes of additional data memory can be addressed by the 8051. The external data memory is accessed using the “MOVX” instruction.
- The 8051's internal data memory is split into three sections: Lower 128 bytes, Upper 128 bytes, and SFRs. While they are physically distinct bodies, the upper addresses and SFRs share the same block of address space, 80H by FFH.
- The upper address space is only accessible via indirect addressing, and SFRs are only accessible via direct addressing, as seen in Higher address space, on the other hand, can be reached using either direct or indirect addressing.



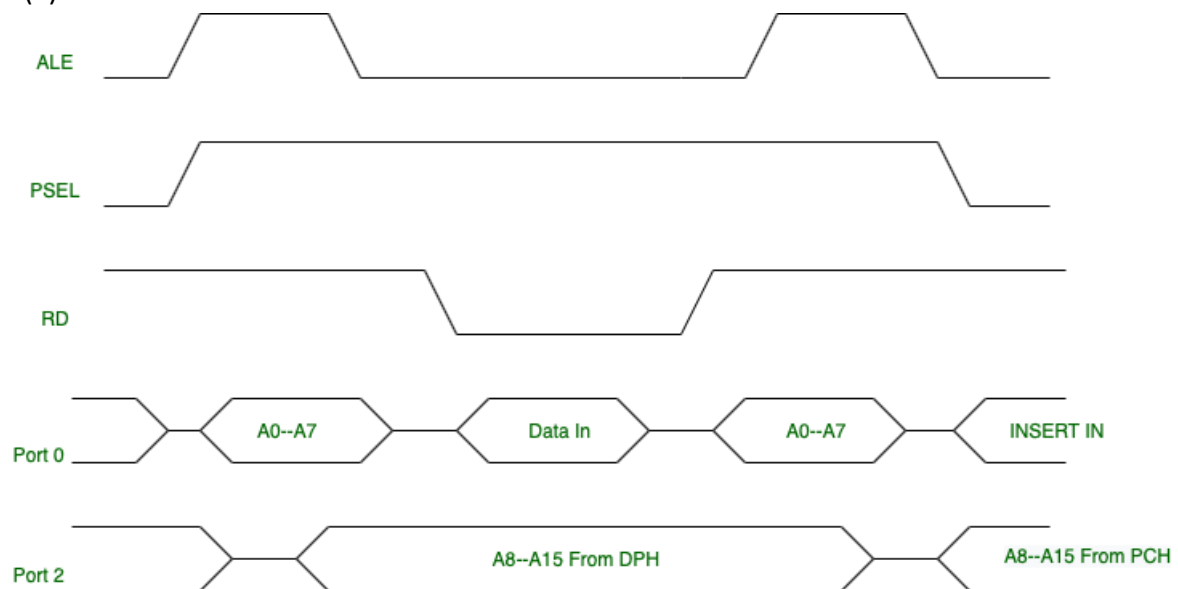
A map of the 8051 data memory Fig. 5

- Fig. 7 a and b show the timing waveform for external data memory read and write cycles, respectively.



timing waveform for external data memory read cycle

7(a)



timing waveform for external data memory write cycle

7(b)

Instructions to Access External Data Memory :

The Table explains the instruction to access external data memory.

Mnemonic	Operation
----------	-----------

MOVX A, @Rp	In this operation, it will copy the contents of the external address in Rp to A.
MOVX A, @DPTR	Copy the contents of the external address in DPTR to A.
MOVX @Rp, A	Copy data from A to the external address in Rp
MOVX DPTR, A	Copy data from A to the external address in DPTR.

Important Points to Remember in Accessing External Memory :

- In the Case of accessing external memory, All external data moves with external RAM or ROM involve the A register.
- While accessing external memory, R can address 256 bytes and DPTR can address 64 k-bytes
- MOV X instruction is used to access external RAM or I/O addresses.

Note -

It must be noted that while the Program counter(PC) will be used then to access external ROM, it will be incremented by 1 (to point to the next instruction) before it is added to A to form the physical address of external ROM.

Memory Address Decoding :

We know that read/write memories consist of an array of registers, in which each register has a unique address: The size of the memory is $N \times M$ as shown in Fig. 11.2.1 (a) where N is the number of registers and M is the word length, in a number of bits.

Example-1 :

If memory is having 12 address lines and 8 data lines, then number of registers/memory

$$\text{locations} = 2^N = 2^{12} = 4096$$

Word length = Mbit

= 8-bit.

Example-2 :

- If the memory has 8192 memory locations, then it has 13 address lines The Table 11.2.1 summarizes the memory capacity and address lines required for memory interfacing

Memory Capacity	Address Line Required
1 K = 1024 memory locations	10
2 K = 2048 memory locations	11
4 K = 4096 memory locations	12

8K= 8192 memory locations	13
16 K = 16384 memory locations	14
32 K = 32768 memory locations	15
64 K = 65536 memory locations	16

- The memory chip has 11 address lines A10-A0, one chip pick (CS), and two control lines, as seen in the table. To allow the output buffer, press RD, and to enable the input buffer, press WR.
- The address lines are decoded using the internal decoder: (b) displays the logic diagram of a 4096 (4 K) register EPROM (Erasable Programmable Read-Only Memory).
- It has a total of 12 address blocks. One chip pick (CS) and one read control signal (A11-A0). EPROM does not need the (WR) signal since it is a read-only memory.
- Memory and I/O modules are used in the microprocessor/microcontroller framework. Since the data, address, and control buses are shared by all devices, the microprocessor can only communicate (read/write) with one device at a time.
- It is essential to decoding the address from the microprocessor/microcontroller in order to connect with memory or I/O devices. The strategies for decoding addresses are described in the following section.
- The memory interfacing requires to following components as follows.
 1. Select the chip.
 2. Identify the register.
 3. Enable the appropriate buffer.

Address Decoding Techniques :

- Absolute decoding/Full decoding
- Linear decoding/Partial decoding

Technique-1 :

Absolute decoding -

- In this technique, all the higher address lines are decoded to select the memory chip, and the memory chip is chosen only for the logic levels defined in these high-order address lines and no other logic levels will select the chip.
- The memory interface with utter encoding is seen in Figure 11.22. In massive memory structures, this addressing strategy is commonly used.

Memory Map -

Memory ICs	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Address
Starting address of EPROM	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000 H
End address of EPROM	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	03FFH
Starting address of RAM	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000 H
End address of RAM	0	0	1	0	0	0	1	1	1	1	1	1	1	1	1	1	23FFH

Technique-2 :

Linear decoding -

- Individual high-order address lines can be used to pick memory chips in compact systems, eliminating the need for hardware for decoding logic. Linear decoding is the term for this method.
- The addressing of RAM using the linear encoding technique is seen in Figure 10. Partial decoding is another name for this process.
- It lowers the cost of encoding and circuitry, but it has the disadvantage of requiring multiple addresses (shadow addresses).
- The addressing of RAM using the linear encoding technique is seen in Figure 10. After inversion, A is a line attached to the chip select signal of the EPROM and then to the chip select signal of the RAM. As a result, EPROM is selected when the status of A line is “empty,” and RAM is selected when the status of the A15 line is “one.” Since

the other address lines aren't used to generate chip pick signals, their status isn't taken into account.

Memory Map -

Memory ICs	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Address
Starting address of EPROM	0	x	x	x	x	x	0	0	0	0	0	0	0	0	0	0	0000 H
End address of EPROM	0	x	x	x	x	x	1	1	1	1	1	1	1	1	1	1	03FFH
Starting address of RAM	1	x	x	x	x	x	0	0	0	0	0	0	0	0	0	0	8000 H
End address of RAM	1	x	x	x	x	x	1	1	1	1	1	1	1	1	1	1	83FFH

Issues in External Memory Interfacing in 8051 Microcontroller :

There are several issues that can arise when interfacing external memory with the 8051 microcontroller. Some common issues include:

1. Timing: The timing of the signals that control the external memory access must be accurate, or it can lead to errors in data transfer or program execution.
2. Interference: Interference from external sources such as electromagnetic interference (EMI) or radio frequency interference (RFI) can cause errors or data corruption in the memory access.

3. **Signal integrity:** The signal integrity of the address and data lines must be maintained to ensure accurate data transfer between the microcontroller and the external memory device.
4. **Address decoding:** The external memory devices must be properly decoded so that the correct memory location is accessed. Incorrect address decoding can lead to data corruption or program errors.
5. **Power supply:** The external memory devices must be powered properly to ensure stable and reliable memory access.
6. **Compatibility:** The external memory devices used for interfacing must be compatible with the microcontroller, including their operating voltage, timing requirements, and interface protocol.
7. **Circuit design:** Proper circuit design is required to ensure that the external memory devices are interfaced correctly with the microcontroller and other components in the system.

3.4 What is the function of ALE pin in 8085?

This pin of the 8085 microprocessor, ALE, is one of the most important pins used in interfacing with external memory devices. During any machine cycle, while interfacing with an external memory using the 8085 microprocessor, the ALE pin latches the lower-order address bits, A0-A7.

During the beginning of each machine cycle, the microprocessor outputs the low-order address bits, A0-A7, onto the address bus. The ALE signal is sent to latch the valid address on the bus; the external devices attached to the microprocessor latch the address on the falling edge of the ALE.

Why ALE Pin is Required in the 8085 MicroProcessor?

The ALE pin is employed for the right synchronization of microprocessor address and data signals with those of external memory gadgets. It facilitates the proper interface of the microprocessor with external memory by providing a clear indication of when the address information is stable and can be used by external devices.

The other key pin on the 8085 microprocessor is the ALE pin, which makes efficient communication between the 8085 microprocessor and the memory devices sitting external to it to carry on all the memory access operations hassle-free.

The 8085 [microprocessor](#) uses the ALE pin to pull the slot at a low during any machine operations it takes while connecting with other computer memories; because it operates by latching the bottom addresses of eight bits (A0-A7) which are output from it. It is for the purpose of maintaining a good level (or point) of control (or flow) within the computer such

that there is consistency between what appears in terms of data as well as where things get stored for reference purposes when using external storage places.

Types of Classification

There are two main types of ALE pins in a microprocessor which depend on how it works.

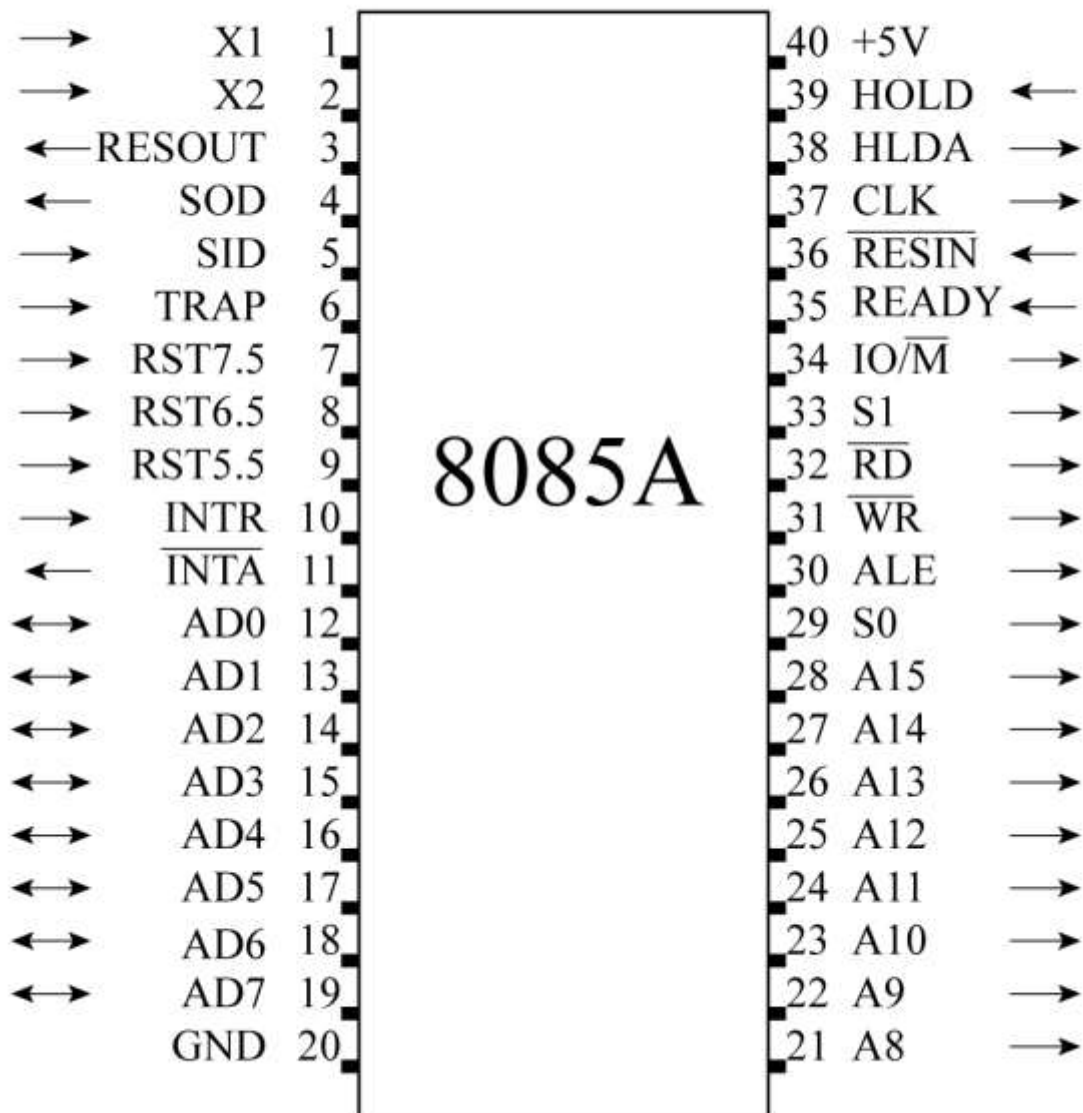
1. ALE Pin Active Low: One is active-high and has been identified as; Here it means that the ALE signal is active at high voltage (usually logic 1). This type contains an indication showing whether the addresses on the bus are good enough for latching by external devices.

2. ALE Pin High Active: In this type, active or valid is considered the ALE signal when it is at a high voltage level usually logic 1. Whenever the ALE pin is high, it means that the address on the bus is irrelevant and cannot be locked by external devices.

These classifications; high-active and low-active help in understanding how ALE pin behaves as regards signaling when address information on the bus should be treated as valid.

Diagram of Latching Circuit with Explanation

Pin Diagram of 8085



Pin Diagram of 8085

We know that AD0 to AD7 lines are multiplexed and the lower half of the address, A0-A7, is available only during T1 of the machine cycle. Now, this lower half of the address is needed during T2 and T3 of the machine cycle also, in order to access a particular location in memory or I/O port. It means that the lower half of an address bus has to be latched in T1 of the machine cycle, so that it is available throughout the machine cycle. To latch falling edge of the address, the 8085 microprocessor uses an external latch with the help of ALE (Address Latch Enable) signal. The figure shows the hardware connection for the latching the lower half of an address. IC 74LS373 contains an 8-bit latch and hence contains 8 D flip flops. Input gets transferred to the output only during clock high. The clock signal gets driven by ALE signal from 8085. The ALE signal is activated only during T1, so input will get transferred to

the output only during T1, i.e. address (A0- A7) on the AD0-AD7 multiplexed bus. In the remaining part of the machine cycle ALE signal is disabled so the output of the latch (A0-A7) remains unchanged. To latch lower half of an address, in each machine cycle, the 8085 gives ALE signal high during T1 of every machine cycle.

The Intel 8085 is an 8-bit microprocessor with 16 address lines designed to give a 16-bit memory location. Out of these 16 address lines, 8 higher order address bits are transmitted on separate eight bit lines and the remaining lower order is sent on another separate eight lines that are multiplexed with the 8 bit data lines. As an Address Enable Latch, ALE becomes a signal that is generated through a positive-going pulse every time there is a new operation initiated by a computer. When pulse goes low, meaning ALE=0, it indicates that the data bus is enabled; and when ALE=1, high pulse means the address bus is enabled.

Working Principle of ALE

A very important part of interfacing with external memory devices on the 8085 microprocessor is played by The address latch enable (ALE) pin. In order to send the lower 8 bits of the address to the memory or an I/O device, the microprocessor activates the ALE pin.

- The lower 8 bits of the memory address are put on the address bus by the microprocessor.
- Next, it turns on ALE. This means that the address on the bus is OK and must be latched by external devices.
- External latching circuits tied to the ALE pin then latch the address on the address bus.
- After the microprocessor takes over address latch, it uses the data bus to transfer data to or from this specific memory location.

The ALE pin of the 8085 microprocessor pin perform specific role in interfacing with external memory devices. For sending lower 8 bits of the address to the memory or I/O device, it activates the ALE pin.

Components of ALE

ALE as a term in the context of the 8085 microprocessor stands for the Address Latch Enable signal (8085). These are some of them on 8085 microprocessor board specific components pertaining ALE

1. Address Bus: An address bus is responsible for carrying memory-address from a microcontroller Unit to other components that include memories and input/output devices.

2. Data Bus: The data bus connects the microprocessor to main memory or I/O peripherals carrying information used in those areas--addresses with instructions or data.

3. Control Signals: Data flow and operations of the microprocessor are governed by various signals referred to as control signals, for example, RD (Read), WR (Write), and ALE (Address Latch Enable).

4. Address Latch: In case ALE signal is active, the address latch becomes a place where addresses from the address bus are captured before being used by external devices.

5. External Memory Interface: The microprocessor has an external memory interface that allows it to interact with external memory devices for data saving and retrieval purposes.

6. Input/output ports: These act like communication channels that facilitate exchange of data between microprocessor and all peripheral devices connected to it.

ALE one of pins instead latches addresses on an external address bus during each machine cycle consequently allowing them (external devices)to know what location should be available next time they want data from it: Arduino.

Construction:

The ALE (Address Latch Enable) pin is an output in the 8085 microprocessor. It has to generate a pulse when the microprocessor intends to latch the lower 8 bits of the memory address.

Physically, the ALE pin connects externally with latch circuits that in turn capture the address applied to the address bus. As part of the address strobe, when the microprocessor activates the ALE pin, a signal is sent out to the external latch circuits connected to the ALE pin, indicating the address on the bus has gone stable and is ready to be latched. Construction of ALE pin: Ensuring that it can generate a reliable pulse to trigger latching at address by external circuits, this functionality is important for proper communication between the microprocessor and external memory or I/O devices.

Important Terminologies

The 8085 microprocessor's ALE (Address Latch Enable) pin can refer to as follows.

ALE Pulse: When an ALE goes from 0-Vcc, it means the lowest eight bits in the memory's address are now validly placed on the address bus where it would subsequently be locked-on inside other electronics facilities.

Address Bus: The group of cables transmitting memory address information between microprocessor & external memory or I/O devices is called Address Bus. Enabling latching of the address, the ALE pin interacts with address bus.

External Latch Circuits: External latch circuits are linked to the ALE pin and are charged with grabbing and grasping onto the last 8 bits of the memory address when ALE makes a pulse.

Address Latching: Address latching describes the way external latch circuits hold onto the bottom eight bits of an address each time a pulse is sent through an external latch on its address enable input (ALE).

Knowing these words makes understanding the ALE pin process by which the information between the microprocessor and external devices about moving an address goes into focus.

Examples

One of the most important signals for the 8085 microprocessor is the Address Latch Enable (ALE) pin. For instance, let's examine how T-States explain the functionality of the ALE pin in relation to microprocessor operation timing.

Imagine a situation in which the 8085 microprocessor is going through one instruction level during which it should work with pin ALE. A signal ALE is produced within a machine cycle so as to latch lower address bits on to an external latch.

This is what the T states structure looks like in simpler terms for the ALE pin operation:

T1 : At the start of each machine cycle, the ALE signal goes high which shows that the microprocessor is almost ready to send out an address into the outside programs circuitry.

T2 : Lower bits of addresses stabilize onto address bus throughout this T-state to help ensure that an appropriate address is available to peripheral equipment.

T3 : Keeping the ALE signal active helps in grabbing the external latch for the lower address byte and maintains it till the end of the machine cycle.

This T states architecture structure of ALE pin operation assists the 8085 microprocessor in aligning the address information hence enabling a proper communication with external memory or devices.

Differences

Its specific function in relation to address latching is what sets the ALE pin apart as regards the other microprocessor pins.

Control signals, data transfer, or power supply may be seen in any other pin except ALE which is there to help in latching lower eight bits of memory address; as such it acts as a bridge between the internal bus system from one side and external memory from another side while enhancing the speed of data transfer simultaneously. This is when it sends out a pulse to show that the address is now stable on the address bus making it possible for external latch circuits to capture it for storage purposes.

It's important to understand that the ALE pin plays a unique role in enabling communication of memory addresses between microprocessor and external devices.

Advantages of ALE Pin

But the ALE pin of microprocessor has several advantages in the context of address handling and interfacing with external devices :

- **Improved Timing Control** : A pulse is produced at proper timing by the ALE pin to latch the lower 8 bits of the memory address, hence providing improved timing control that makes sure the external latch circuits capture the address information accurately.
- **Simplified Interface Design** : The microprocessor can easily communicate the lower 8 bits of the address to the external devices using the ALE pin itself, avoiding complex address decoding circuits. This simplifies the interface design between the microprocessor and external memory or i/o devices.
- **ALE allows efficient use of the address bus** : The ALE pin is a facility to indicate that the lower order address bits can be latched, that is, it is informing that this is stable. It thus enhances overall performance for efficient utilization of the address bus.
- **Flexibility in System Configuration** : With the availability of the ALE pin, a great deal of flexibility in system configuration is afforded by possibly latching the address information by external devices whenever required. This flexibility is very useful while interfacing many peripherals or modules of memory.
- **Reliability** : The presence of an ALE pin helps in reliable communication of the address information from the microprocessor to the external component, thus reducing the possibility of data errors or corruption during address transmission.

These advantages underpin the central role that the ALE pin plays in issues concerning handling, and then to the general efficiency and reliability of the microprocessor system.

Disadvantages of ALE Pin

Even though a microprocessor's ALE pin has three primary advantages when dealing with addresses, this does not happen sans associated limitations or disadvantages:

- **Complexity in Circuitry**: The addition of an ALE pin to capture and hold the bits at the lower address requires additional external latch circuits. Thus, this structure increases the system design complexity and the count of components.
- **Timing Constraints**: The ALE signal should be precisely timed to latch address information; otherwise, it will not correctly latch. The occurrence of any type of timing fault or delay in generating an ALE pulse may result in data integrity issues or even address decoding errors.
- **Potential for Signal Glitches**: When the ALE signal does not align perfectly with the address bus signals, this can result in signal glitches during address transitions and noise. This may ultimately affect the reliability of transfers conveying information about addresses.

- **Limited Addressing Range:** Typically, the ALE pin latches only the lower 8 bits of the memory address. Lots of other mechanisms or signals may be needed, especially in a large address space, to handle upper address bits—making addressing schemes more complex.
- **Dependency on External Components:** The operation of the ALE pin is dependent on the external latch circuits to capture the address information. Failures or malfunctioning in parts connected externally, like these, may affect the latching of an address properly.

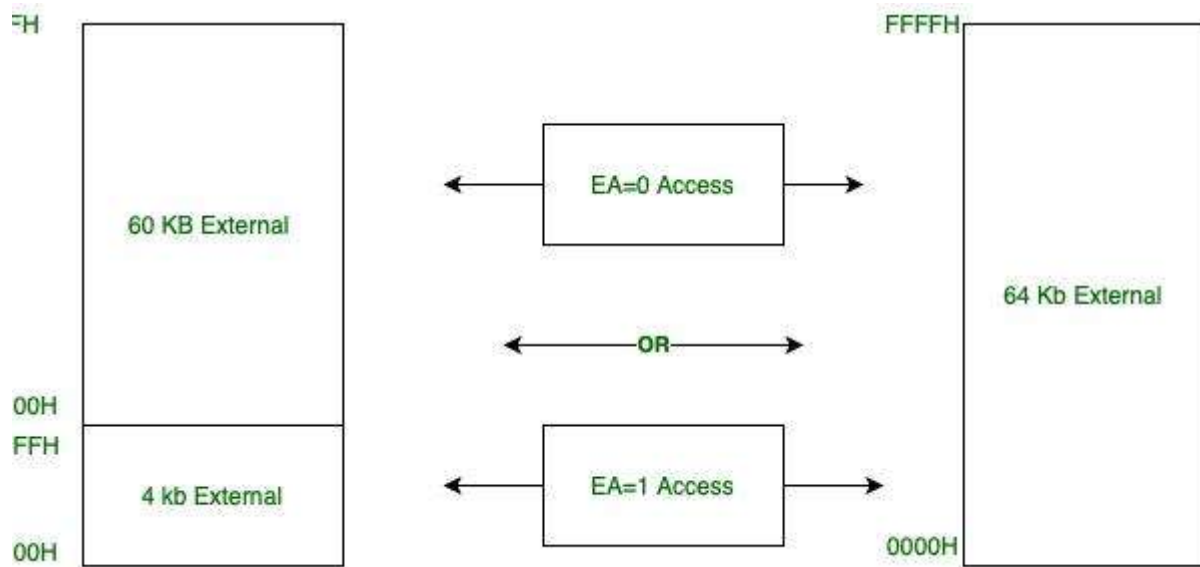
Understanding this disadvantage should be able to help weigh the trade-offs associated with the use of the ALE pin in microprocessor systems.

Applications of ALE

The ALE pin of a microprocessor finds applications in several other scenarios where precise address handling and interfacing with other external devices are required. The common applications of the ALE pin include :

- **Memory Interfacing:** Since the microprocessor has to access external memory devices in some systems, the ALE pin is used to latch the lower address bits for selecting the memory location. This would help in correct addressing of memory locations and quick retrieval of data from them.
- **I/O Device Interface:** The ALE pin permits latching of the address information for accessing certain registers or ports installed on the external devices when input-output devices are interfaced to the microprocessor. It will also permit undisturbed communication between the microprocessor and peripherals.
- **Multiplexed Address Bus:** In some microprocessors, address and data share the same lines, and the ALE pin is used to de-multiplex the address and data. This means that the address information is separated through this pin so that it can be understood by external devices.
- **DMA:** In which this ALE pin is used to transmit the memory address information precisely to the [DMA controller](#) while performing a direct data transfer from peripherals into the memory and vice versa by means of DMA controllers, that are outside the intervention of any CPU within the process.
- **System Control Signals:** The ALE pin can also be used to generate control signals for various system operations based on the address information. For instance, it can trigger some action or functionality of the system depending on the accessed memory address.

3.5 Draw the memory interfacing of the 8K byte EPROM and 8K byte RAM with starting address 6000H and 7000h respectively.



To draw the memory interface for an 8KB EPROM ([starting 6000H](#)) and 8KB RAM ([starting 7000H](#)), you'll use address lines A0-A12 for each chip, A13-A15 for chip select ([CS](#)) via a decoder, [Data Bus D0-D7](#) for both, and [control signals](#) (RD', [WR'](#), [IO/M'](#) for an 8085), with the EPROM needing [Read Only](#) access and RAM for Read/Write, requiring careful decoding for distinct address ranges.

1. Memory Map & Address Lines

- **8K EPROM:** Needs 13 address lines (A0-A12) (since

$$2^{13} = 8192$$

213=8192

).

- Range: 6000H to 7FFFH (6000H + 1FFFH).
- **8K RAM:** Needs 13 address lines (A0-A12).
 - Range: 7000H to 8FFFH (7000H + 1FFFH).
- **Address Lines Used:**
 - **A0-A12:** Connect to the memory chips (e.g., A0-A12 to EPROM, A0-A12 to RAM).
 - **A13, A14, A15:** Used for decoding the CS signals.

2. Address Decoding Logic (Using 3-to-8 Decoder like 74LS138)

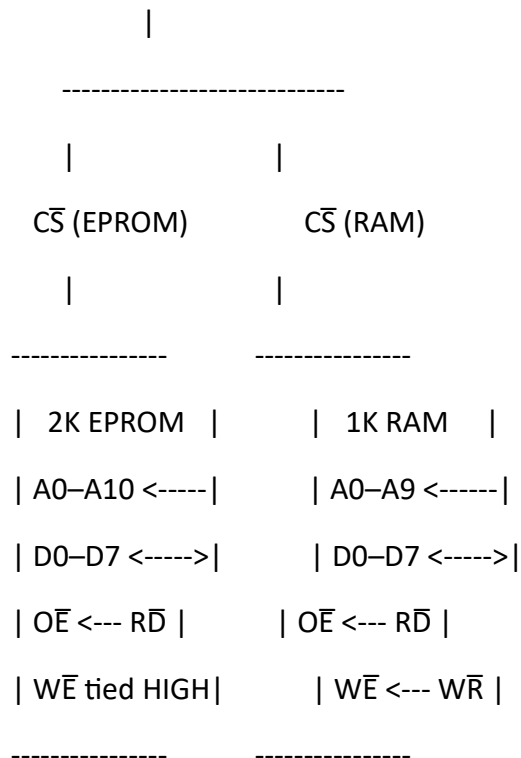
- **Decoder Inputs (C, B, A):** Use A15, A14, A13 from the CPU.
- **Decoder Outputs (Y0-Y7):** One output activates a chip select.
- **Chip Select Logic:**
 - **EPROM (6000H-7FFFH):** A15=0, A14=1, A13=1 (binary 011). Connect Y3 to EPROM's CS (active low).
 - **RAM (7000H-8FFFH):** A15=1, A14=0, A13=0 (binary 100). Connect Y4 to RAM's CS (active low).
 - **Decoder Enable:** Ground E1' and E2', connect E3 to Vcc (+5V) to enable the decoder.

3. Connections

- **Address Bus (A0-A15):** A0-A12 to both chips; A13-A15 to the decoder.
- **Data Bus (D0-D7):** Connect to both EPROM and RAM data pins.
- **Control Signals (8085):**
 - IO/M' (Low for Memory): Connect to the decoder's enable or directly to chip selects (as needed).
 - RD' (Memory Read): Connect to EPROM's OE (Output Enable) and RAM's OE.
 - WR' (Memory Write): Connect to RAM's WE (Write Enable); EPROM doesn't use it.

4. Block Diagram (Conceptual)

1. **CPU (8085)** -> Address Bus (A0-A15) & Data Bus (D0-D7) & Control Lines (RD', WR', IO/M')
2. **A0-A12** -> EPROM & RAM Address Pins
3. **A13-A15** -> 3-to-8 Decoder Inputs (C, B, A)
4. **Decoder Outputs (Y3, Y4)** -> EPROM CS' & RAM CS'
5. **D0-D7** -> EPROM & RAM Data Pins
6. **RD'** -> EPROM OE & RAM OE
7. **WR'** -> RAM WE



Control Signal Connections

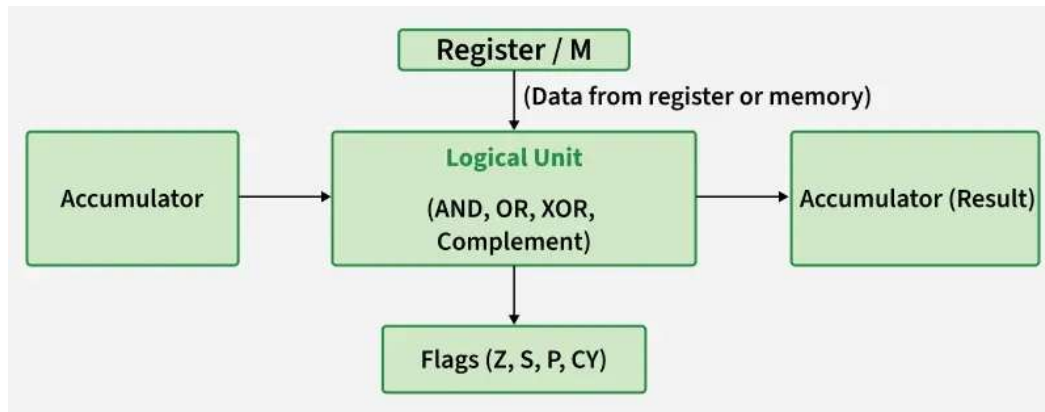
- **EPROM**
 - $OE \rightarrow RD$
 - $WE \rightarrow V_{cc}$ (disabled, since EPROM is read-only)
- **RAM**
 - $OE \rightarrow RD$
 - $WE \rightarrow WR$

Explanation

The 8085 uses a multiplexed address/data bus. The lower address lines are latched using ALE (assumed). Address lines A0–A10 are connected to the EPROM to access 2K locations, while A0–A9 are connected to the RAM for 1K locations. Higher address lines are decoded using external logic to generate chip select signals. EPROM is used for program storage, and RAM is used for data storage.

3.7 Explain the logical instruction in detail.

Logical instructions in the 8085 microprocessor are a set of instructions that perform logical operations on data in registers and memory. Logical operations are operations that manipulate the bits of data without affecting their numerical value. These operations include AND, OR, XOR, and NOT.



Logical Operations

Common Logical Instructions

The logical instructions in the 8085 microprocessor include:

ANA – Logical AND

Performs a bitwise AND between the accumulator and a specified register or memory location, storing the result in the accumulator.

- The Zero (Z), Sign (S), Parity (P), and Carry (CY) flags are affected (CY is always reset).
- Commonly used for masking specific bits in the accumulator.

ORA – Logical OR

Performs a bitwise OR between the accumulator and a specified register or memory location, storing the result in the accumulator.

- Z, S, P, and CY flags are affected (CY is always reset).
- Useful for setting specific bits in the accumulator without affecting others.

XRA – Logical XOR

Performs a bitwise Exclusive OR between the accumulator and a specified register or memory location, storing the result in the accumulator.

- Z, S, P, and CY flags are affected (CY is always reset).
- Used to toggle specific bits or clear the accumulator when XORed with itself.

CPL – Logical Complement

Flips all bits of the accumulator, inverting its value.

- No flags are affected.
- Useful for bitwise inversion operations.

CMA – Complement Accumulator

Performs a bitwise complement of the accumulator, flipping all bits.

- No flags are affected.
- Often used in two's complement or logical negation operations.

Usage of Logical instructions in 8085 microprocessor

Here are some specific reasons why logical instructions are used in the 8085 microprocessor:

1. **Data comparison:** Logical instructions can be used to compare two pieces of data bit-by-bit. For example, the XOR instruction can be used to compare the contents of two registers or memory locations and set a flag if they are not equal.
2. **Data masking:** Logical instructions can be used to selectively mask or extract specific bits of data. For example, the AND instruction can be used to mask off all but a specific set of bits, while the OR instruction can be used to set specific bits to 1.
3. **Data encryption:** Logical instructions can be used to perform bitwise encryption of data. By performing a logical XOR between the data and a secret key, the data can be encrypted in a reversible way that can only be decrypted with the same key.
4. **Decision-making:** Logical instructions can be used to make decisions in programs. For example, the AND instruction can be used to test if multiple conditions are true, while the OR instruction can be used to test if at least one condition is true.

Table of Instructions with Opcodes

Following is the table showing the list of logical instructions:

In the table,

R stands for register, M stands for memory and Mc stands for memory contents.

OPCODE and OPERAND	DESTINATION and EXAMPLE
ANA R	$A = A \text{ AND } R \rightarrow \text{ANA } B$
ANA M	$A = A \text{ AND } M_c \rightarrow \text{ANA } 2050$
ANI 8-bit data	$A = A \text{ AND } 8\text{-bit data} \rightarrow \text{ANI } 50$

ORA R	A = A OR R → ORA B
ORA M	A = A OR Mc → ORA 2050
ORI 8-bit data	A = A OR 8-bit data → ORI 50
XRA R	A = A XOR R → XRA B
XRA M	A = A XOR Mc → XRA 2050
XRI 8-bit data	A = A XOR 8-bit data → XRI 50
CMA	A = 1's complement of A → CMA
CMP R	Compare R with A and set flags → CMP B
CMP M	Compare Mc with A and set flags → CMP 2050
CPI 8-bit data	Compare 8-bit data with A and set flags → CPI 50
RRC	Rotate accumulator right (no carry) → RRC
RLC	Rotate accumulator left (no carry) → RLC
RAR	Rotate accumulator right with carry → RAR
RAL	Rotate accumulator left with carry → RAL
CMC	Complement the carry flag → CMC
STC	Set the carry flag → STC

3.8 Can you describe how the addition instruction with carry operates?

Arithmetic instructions in the 8085 microprocessor allow it to perform operations such as addition, subtraction, increment, decrement, and related tasks on data stored in registers, memory, or immediate values.

Types of Arithmetic Instructions

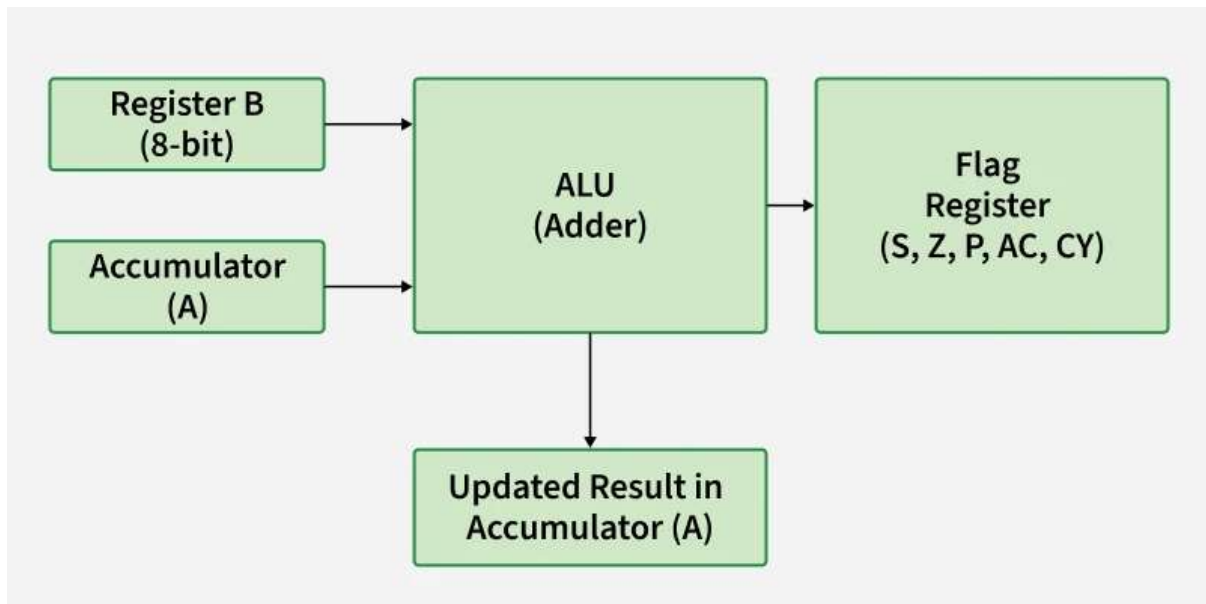
The arithmetic instructions can be broadly classified into the following types:

1. Addition Instructions

Addition is a fundamental arithmetic operation used extensively. The 8085 microprocessor supports various types of addition:

- **ADD r:** Adds the content of register r (B, C, D, E, H, L, or A) to the accumulator (A).
Example: ADD B adds the content of register B to accumulator A.

- **ADI data:** Adds an 8-bit immediate data to the accumulator. Example: ADI 25H adds the hexadecimal value 25H to A.
- **ADC r:** Adds the content of register r plus the carry flag to the accumulator. Used for multi-byte addition where carry propagation is necessary.
- **ACI data:** Adds immediate data and the carry flag to the accumulator.
- **ADD M:** Adds the content of the memory location pointed to by the HL pair to the accumulator.



Add B execution in ALU

2. Subtraction Instructions

These instructions allow subtraction operations, often used in loops, comparisons, and arithmetic calculations:

- **SUB r:** Subtracts the content of register r from the accumulator.
- **SUI data:** Subtracts immediate 8-bit data from the accumulator.
- **SBB r:** Subtracts the content of register r and the borrow (carry) flag from the accumulator.
- **SBI data:** Subtracts immediate data and the borrow flag from the accumulator.
- **SUB M:** Subtracts the content of memory pointed by HL from the accumulator.

3. Increment and Decrement Instructions

These instructions increase or decrease the value of registers or memory locations by 1:

- **INR r:** Increment the content of register r by 1.

- **DCR r:** Decrement the content of register r by 1.
- **INR M:** Increment the content of the memory location pointed by HL.
- **DCR M:** Decrement the content of the memory location pointed by HL.

4. Decimal Adjust Accumulator (DAA)

The 8085 microprocessor can operate on BCD (Binary Coded Decimal) numbers. After addition of two BCD numbers, the result in the accumulator may not be in valid BCD format. The DAA instruction adjusts the accumulator to produce a valid BCD number.

5. Rotate and Other Arithmetic-Related Instructions

Although primarily used for bit manipulation, rotate instructions also assist arithmetic operations such as multiplication or division by powers of 2:

- **RLC (Rotate Left Accumulator)**
- **RRC (Rotate Right Accumulator)**
- **RAL (Rotate Left through Carry)**
- **RAR (Rotate Right through Carry)**

Flags Affected by Arithmetic Instructions

Most arithmetic instructions affect the following flags in the 8085 flag register:

- **Sign (S)**
- **Zero (Z)**
- **Auxiliary Carry (AC)**
- **Parity (P)**
- **Carry (CY)**

For example, addition and subtraction instructions set or reset the carry flag depending on overflow, set zero flag if the result is zero, etc.

Various Arithmetic Instructions

Following are arithmetic instructions with example, which can be executed in 8085 microprocessor:

Opcode & Operand	Explanation & Example
ADD R	$A = A + R$ ADD B $\rightarrow A = A + B$
ADI 8-bit data	$A = A + \text{Immediate 8-bit data}$ ADI 50H $\rightarrow A = A + 50H$
ADC R	$A = A + R + \text{Previous Carry}$ ADC B $\rightarrow A = A + B + CY$
ACI 8-bit data	$A = A + \text{Immediate 8-bit data} + \text{Previous Carry}$ ACI 50H
SUB R	$A = A - R$ SUB B $\rightarrow A = A - B$
SUI 8-bit data	$A = A - \text{Immediate 8-bit data}$ SUI 50H
SBB R	$A = A - R - \text{Previous Carry}$ SBB B $\rightarrow A = A - B - CY$
SBI 8-bit data	$A = A - \text{Immediate 8-bit data} - \text{Previous Carry}$ SBI 50H
INR R	$R = R + 1$ INR B $\rightarrow B = B + 1$
INX rp	Register Pair = Register Pair + 1 INX H $\rightarrow HL = HL + 1$
DCR R	$R = R - 1$ DCR B $\rightarrow B = B - 1$
DCX rp	Register Pair = Register Pair - 1 DCX H $\rightarrow HL = HL - 1$
DAD rp	$HL = HL + \text{Register Pair}$ DAD H $\rightarrow HL = HL + HL$

3.9 Which are the control instructions in 8085? Discuss it.

Microprocessors are electronic devices that process digital information using instructions stored in memory. Machine control instructions are a type of instruction that control machine functions such as Halt, Interrupt, or do nothing. These instructions alter the different type of operations executed in the processor. In this article, we will discuss the various types of machine control instructions found in microprocessors and their significance in controlling the microprocessor's operations.

Types of Machine Control Instructions

1. NOP (No operation)
2. HLT (Halt)
3. DI (Disable interrupts)
4. EI (Enable interrupts)
5. SIM (Set interrupt mask)
6. RIM (Reset interrupt mask)

1. NOP (No operation)

Opcode- 00

Operand- None

Length- 1 byte

M-Cycles- 1

T-states- 4

Hex code- 00

NOP is used when no operation is performed. It is commonly used to fill in time delay or to delete and insert instructions while troubleshooting. During the execution of NOP, no flags are affected.

2. HLT (Halt)

Opcode- 76

Operand- None

Length- 1 byte

M-Cycles- 2 or more

T-states- 5 or more

Hex code- 76

HLT is used to stop the execution of the program temporarily. The microprocessor finishes executing the current instruction and halts any further execution. The contents of the registers are unaffected during the HLT state. HLT can be used to enter a wait state in which the microprocessor waits for a specific event to occur before resuming execution.

3. DI (Disable interrupts)

Opcode- F3

Operand- None

Length- 1 byte

M-Cycles- 1

T-states- 4

Hex code- F3

DI is used when the execution of a code sequence cannot be interrupted. For example, in critical time delays, this instruction is used at the beginning of the code and the interrupts are enabled at the end of the code. The TRAP interrupt cannot be disabled.

4. EI (Enable interrupts)

Opcode- FB

Operand- None

Length- 1 byte

M-Cycles- 1

T-states- 4

Hex code- FB

EI is used to enable interrupts after a system reset or the acknowledgement of an interrupt. The Interrupt Enable flip-flop is reset, thus disabling the interrupts.

5. SIM (Set interrupt mask)

Opcode- 30

Operand- None

Length- 1 byte

M-Cycles- 1

T-states- 4

Hex code- 30

SIM is used for the implementation of different interrupts of 8085 microprocessor like RST 7.5, 6.5, and 5.5 and also serial data output. It does not affect the TRAP interrupt.

6. RIM (Reset interrupt mask)

Opcode- 20

Operand- None

Length- 1 byte

M-Cycles- 1

T-states- 4

Hex code- 20

RIM is a multipurpose instruction used to read the status of 8085 interrupts 7.5, 6.5, 5.5, and to read serial data input bit.

Application of Machine Control Instructions

Machine control instructions are used in a variety of applications, including embedded systems, control systems and industrial automation.

- In embedded systems, machine control instructions are used to control the microprocessor's operations, including the timing and sequencing of instructions.
- In control systems, machine control instructions are used to monitor and control physical processes, such as temperature, pressure, and flow.
- In industrial automation, machine control instructions are used to control robots, assembly lines, and other machines.

Features of Machine Control Instructions

- **Halting Execution:** The HLT (Halt) instruction stops the execution of further instructions, putting the microprocessor in an idle state until an external interrupt or reset occurs.
- **No Operation Execution:** The NOP (No Operation) instruction does not perform any action but advances the instruction pointer, often used for timing adjustments and debugging.
- **Interrupt Control:** The EI (Enable Interrupts) instruction allows the microprocessor to recognize and respond to external interrupts. The DI (Disable Interrupts) instruction prevents the microprocessor from processing interrupts, useful for critical sections where interrupt handling should be paused.
- **Processor State Management:** These instructions can alter the processor's state by enabling or disabling functionalities like interrupts and execution flow control.

Limitations of Machine Control Instructions

- **HLT instruction** can cause the microprocessor to enter a wait state indefinitely, affecting system responsiveness.
- **NOP instruction** consumes clock cycles without performing useful work, leading to inefficiencies in processing.
- **DI instruction** can disable interrupts temporarily, but improper use may cause the system to miss critical interrupts.
- **EI instruction** enables interrupts, but when multiple interrupts are received simultaneously, it can lead to unpredictable system behavior.